158.337
Database Development

Semester-1

Week 9

Introduction to LINQ

LINQ - Language Integrated Query

Object-oriented (OO) concepts involve classes, methods, objects, etc.

LINQ integrates OO technology with information that is not defined in an OO manner like table structures used in relational databases.

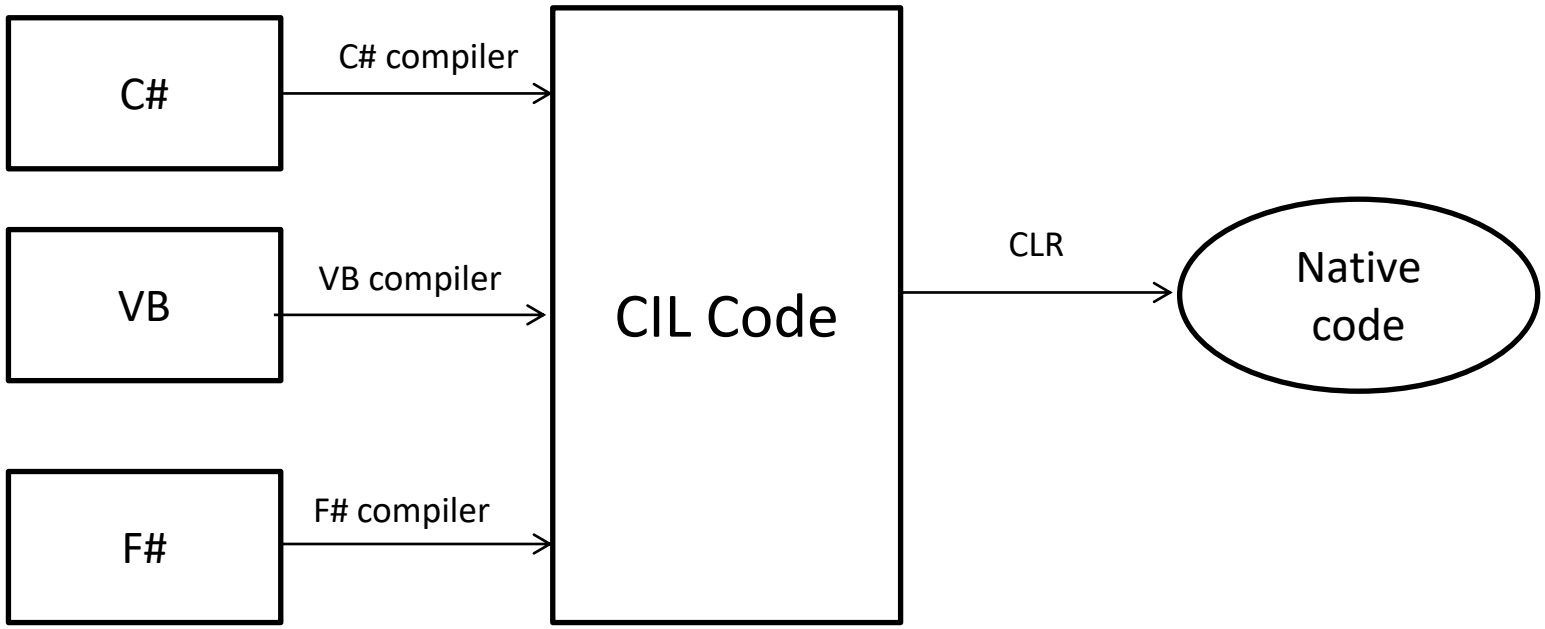
LINQ provides us with new query capabilities by using the language syntax of C# and Visual Basic.

LINQ - Language Integrated Query

Programming statements using LINQ can be checked at compile-time unlike statements using SQL, i.e. it uses a unified query syntax regardless of the source of data (Oracle, Access, etc)

LINQ integrates SQL schema directly into the CLR (Common Language Runtime) metadata of the .NET framework. So, the schema is directly accessible from within the programming environment.

Native Code


C#
C# compiler
VB
VB compiler
F#
F# compiler
CIL Code
CLR
Native code

Software Tool

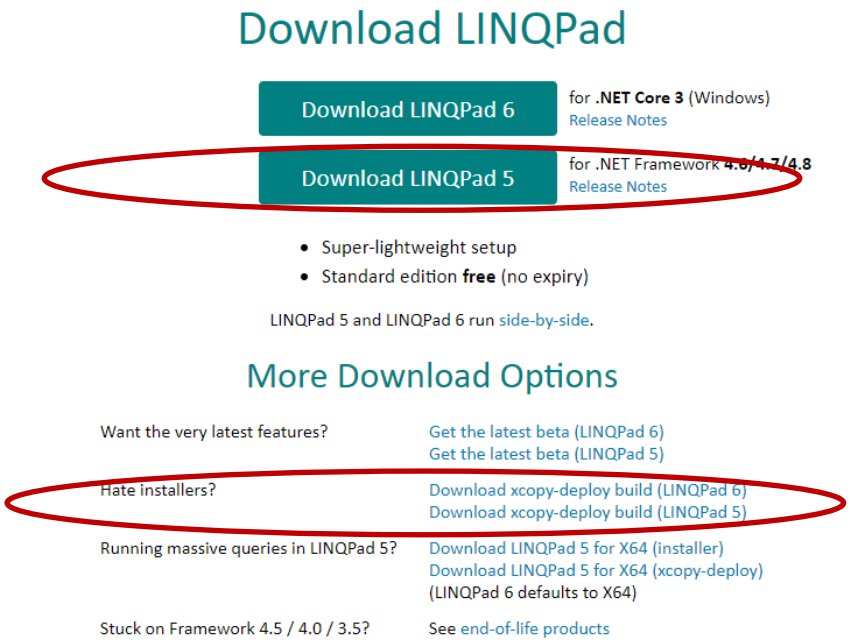
Use LINQPad 5

Download from https://www.linqpad.net/

Two options of downloading: A or B

A

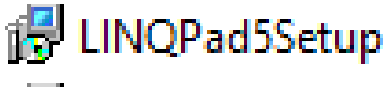
B

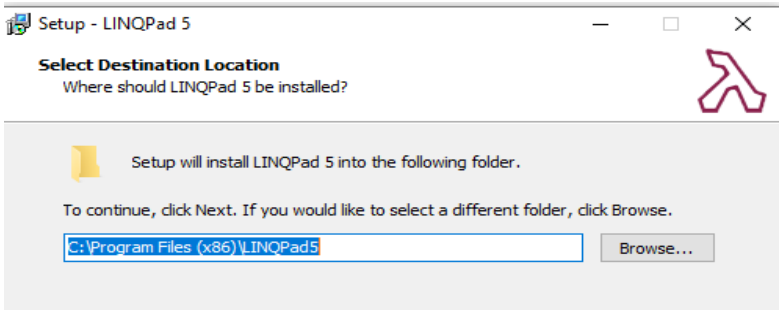


Use Option B

Software Tool

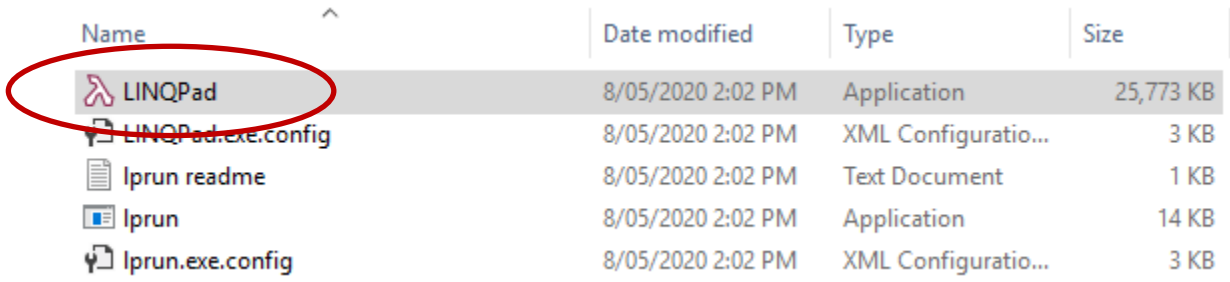
Option A: A setup file that allows you to install





Option B: Extract files and run

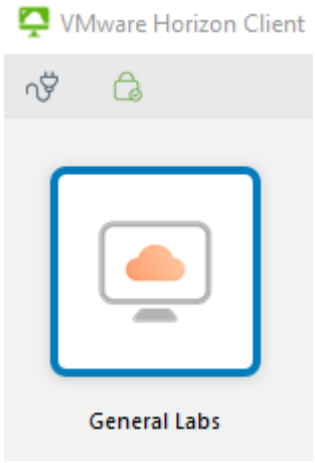


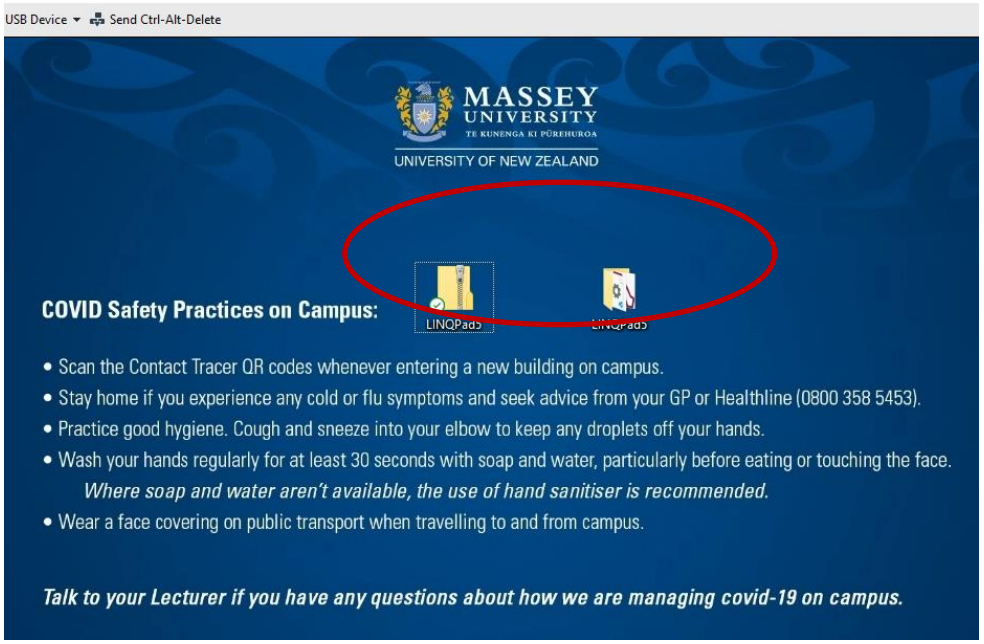


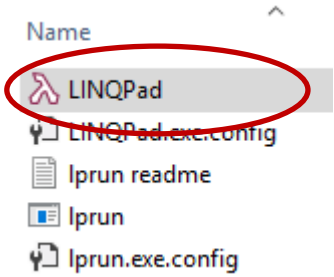
Name	Date modified	Type	Size
LINQPad	8/05/2020 2:02 PM	Application	25,773 KB
LINQPad.exe.config	8/05/2020 2:02 PM	XML Configuratio...	3 KB
lprun readme	8/05/2020 2:02 PM	Text Document	1 KB
lprun	8/05/2020 2:02 PM	Application	14 KB
lprun.exe.config	8/05/2020 2:02 PM	XML Configuratio...	3 KB

Using LinqPad5 in VMWare

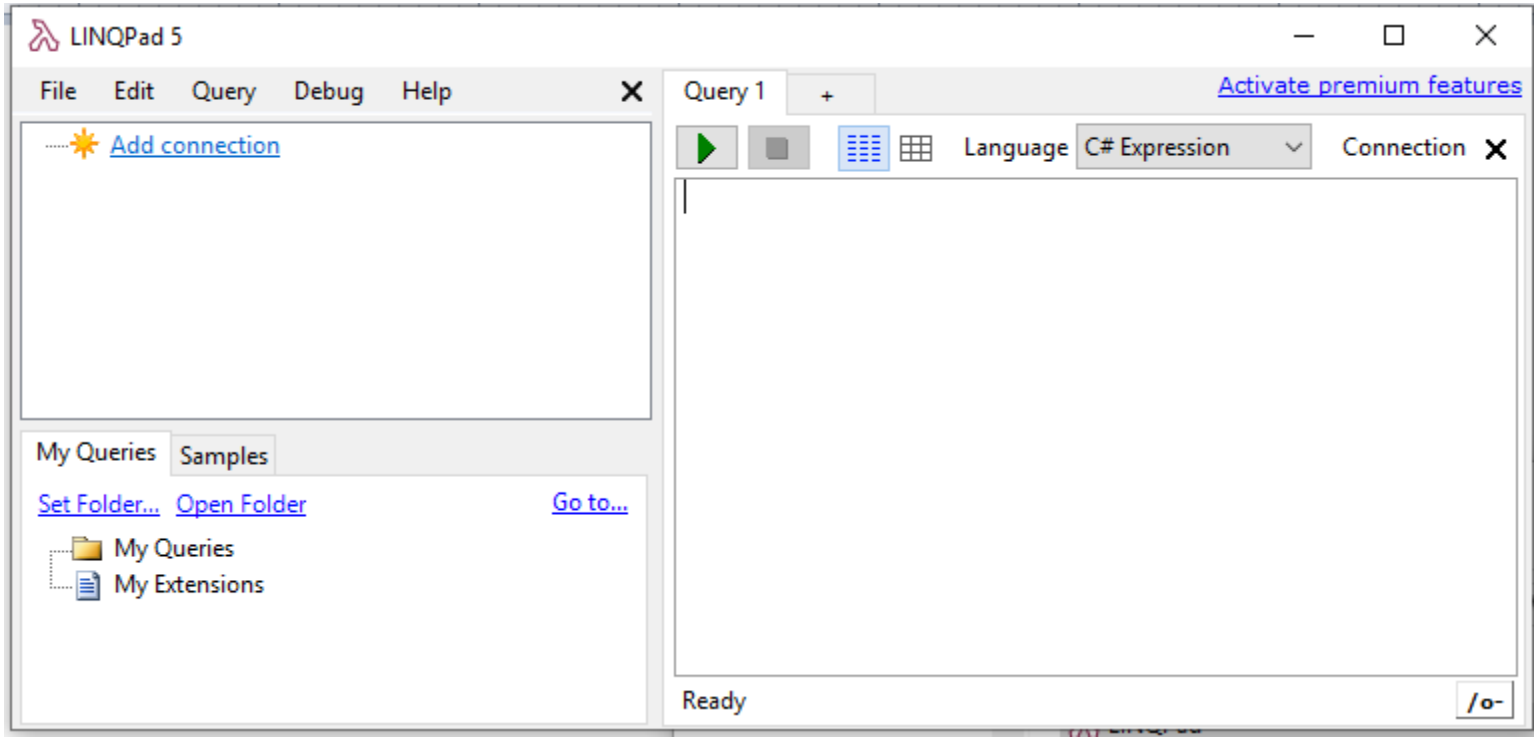
You will not be allowed to setup a file via VMWare Horizon. Copy the LinQPad5.zip folder into the Remote Desktop provided via General labs. Extract this folder. Rune the exe file







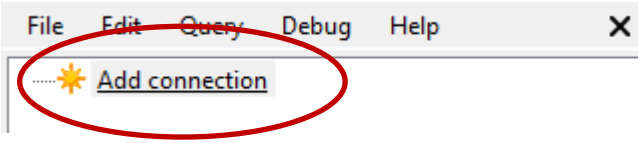
LINQPad 5



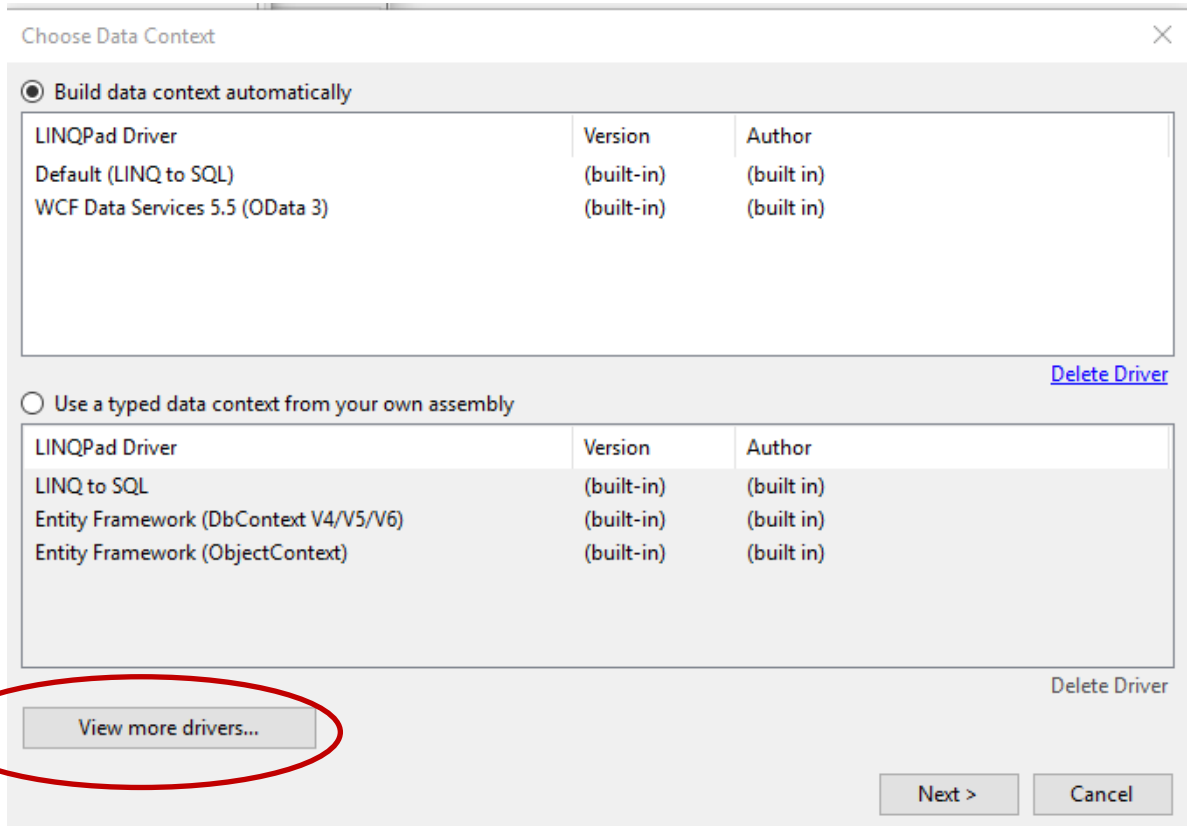
- Add a connection for connecting to your Oracle account.

Connecting to Oracle Server

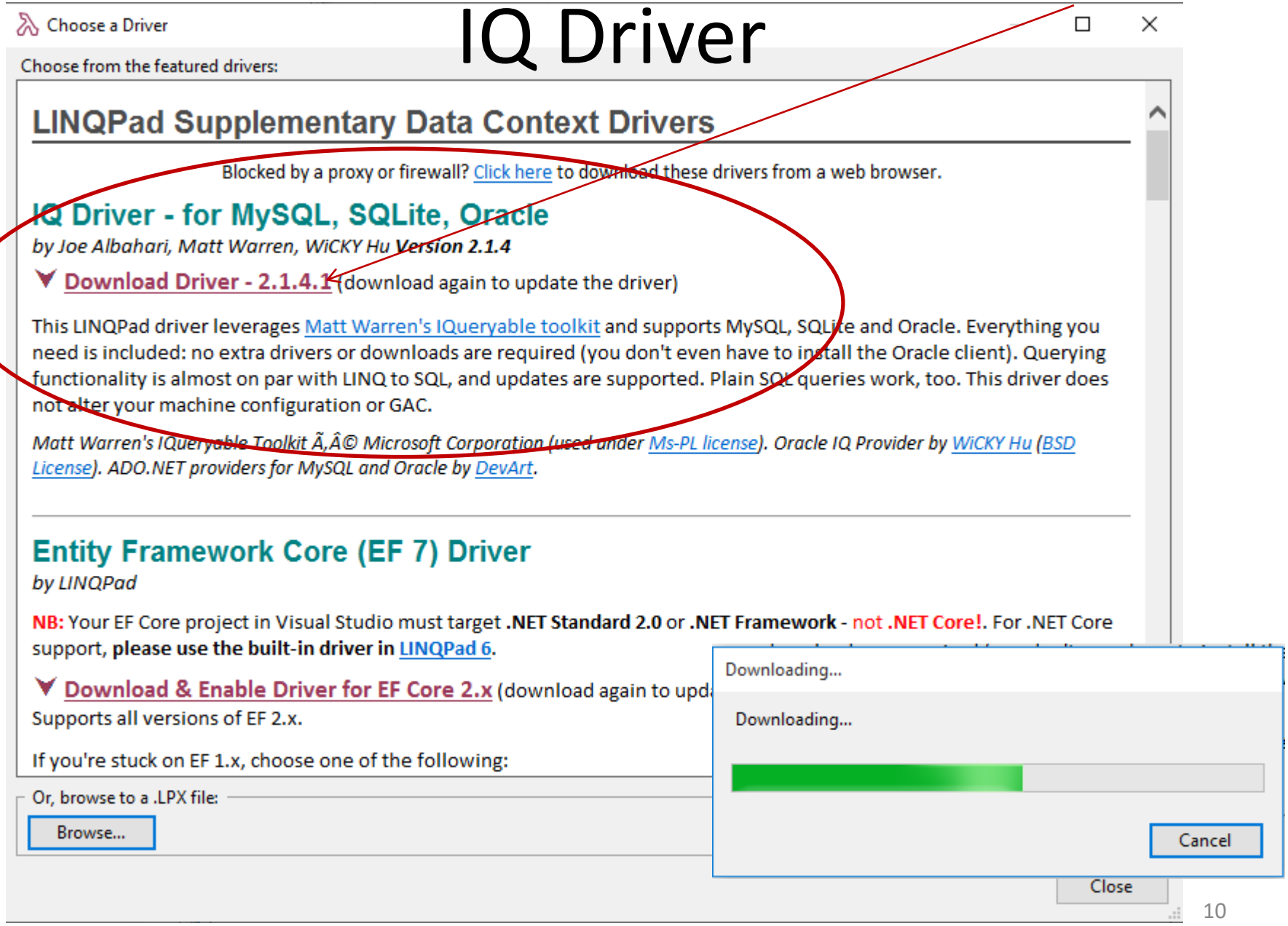
First Step – ADD connection.



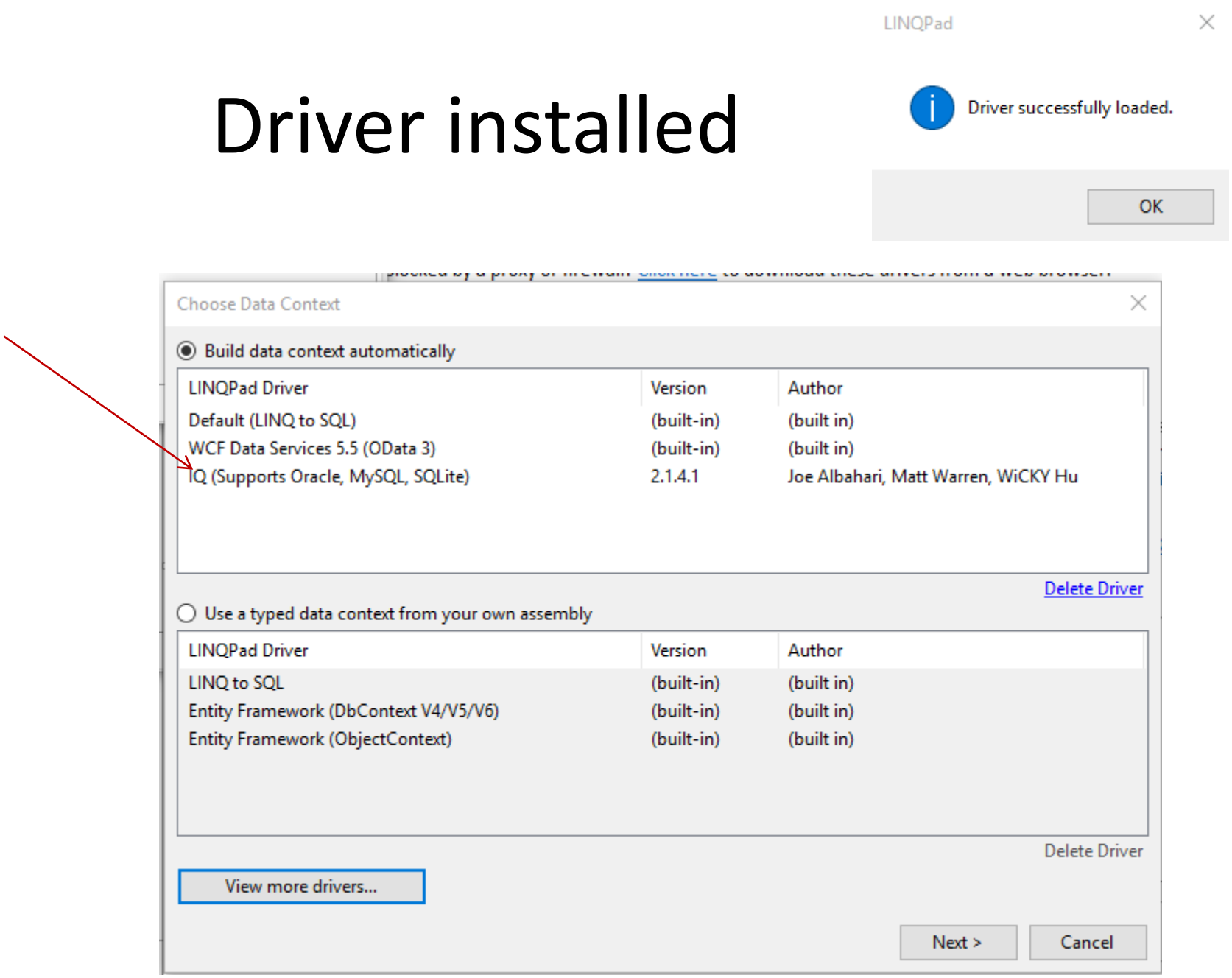
Second Step - Click – View more drivers



IQ Driver

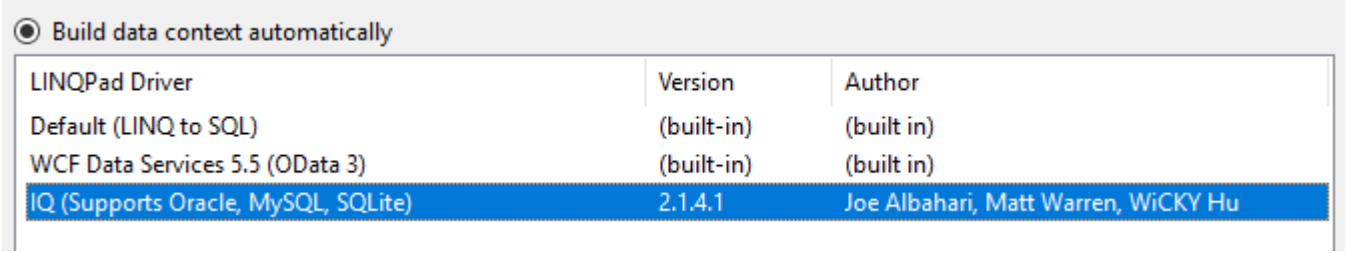

Choose a Driver
Choose from the featured drivers:
LINQPad Supplementary Data Context Drivers
Blocked by a proxy or firewall? Click here to download these drivers from a web browser.
IQ Driver - for MySQL, SQLite, Oracle
by Joe Albahari, Matt Warren, WiCKY Hu Version 2.1.4
Download Driver - 2.1.4.1 (download again to update the driver)
This LINQPad driver leverages Matt Warren's IQueryable toolkit and supports MySQL, SQLite and Oracle. Everything you need is included: no extra drivers or downloads are required (you don't even have to install the Oracle client). Querying functionality is almost on par with LINQ to SQL, and updates are supported. Plain SQL queries work, too. This driver does not alter your machine configuration or GAC.
Matt Warren's IQueryable Toolkit Ã‚Â© Microsoft Corporation (used under Ms-PL license). Oracle IQ Provider by WiCKY Hu (BSD License). ADO.NET providers for MySQL and Oracle by DevArt.
Entity Framework Core (EF 7) Driver
by LINQPad
NB: Your EF Core project in Visual Studio must target .NET Standard 2.0 or .NET Framework - not .NET Core!. For .NET Core support, please use the built-in driver in LINQPad 6.
Download & Enable Driver for EF Core 2.x (download again to upd
Supports all versions of EF 2.x.
If you're stuck on EF 1.x, choose one of the following:
Or, browse to a .LPX file:
Browse...
Close
Downloading...
Downloading...
Cancel

Driver installed



Connection Configuration

Select IQ

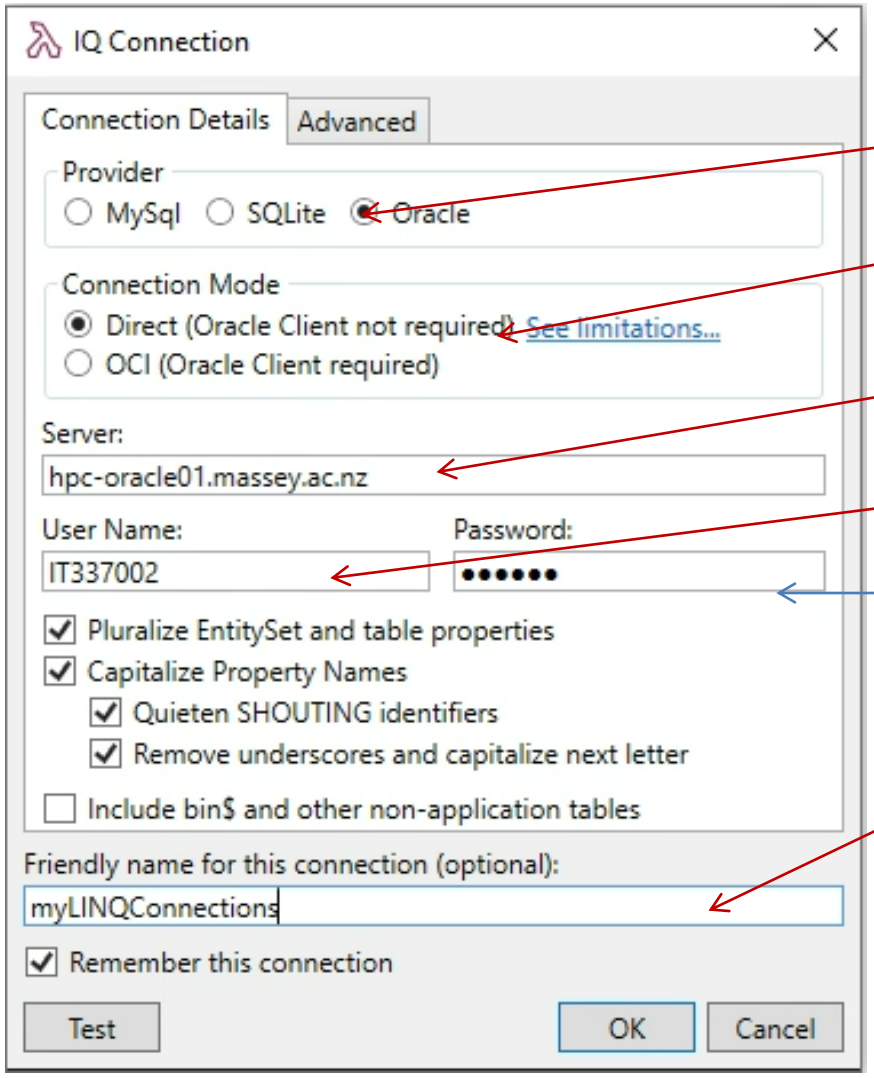


Next

- Step 1 - In Tab Connection Details

- Step 2 – In Advanced

Connection (step 1)



Oracle

Direct

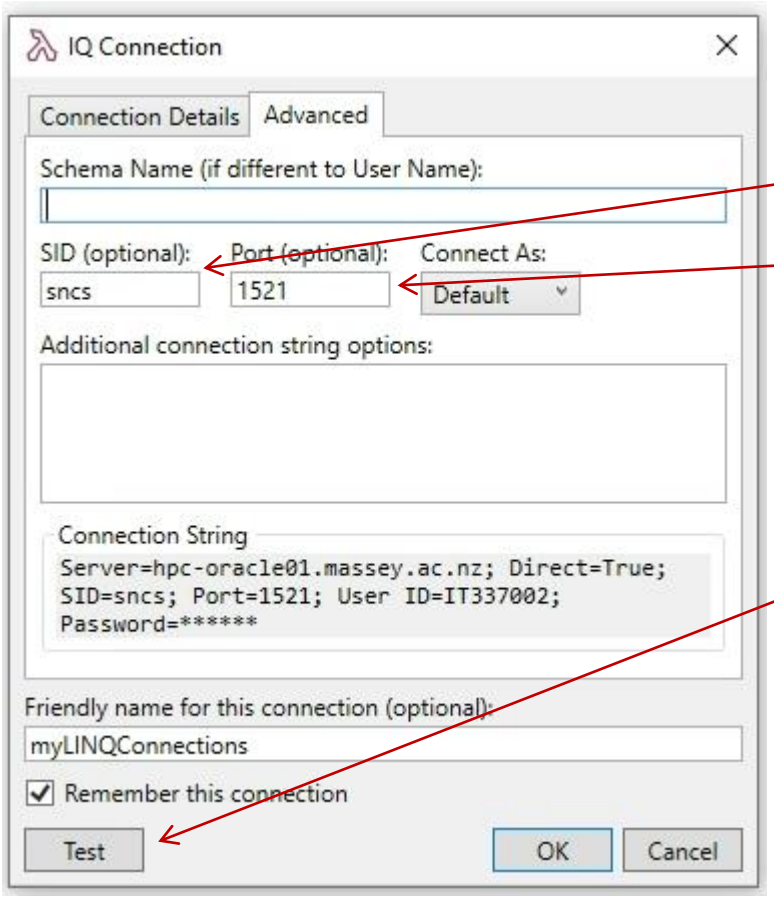
Server:

Username

Password

ConnectionName

Connection (step 2)

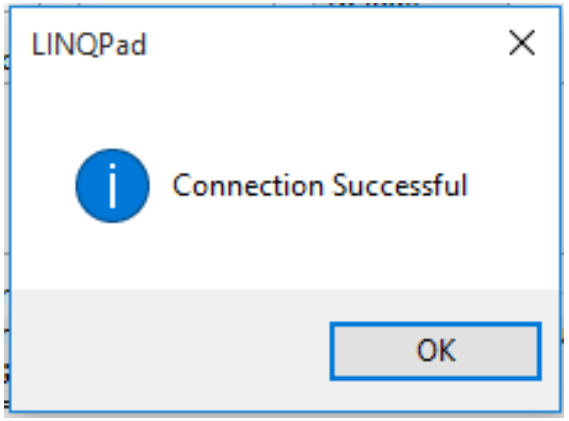


sncs

1521

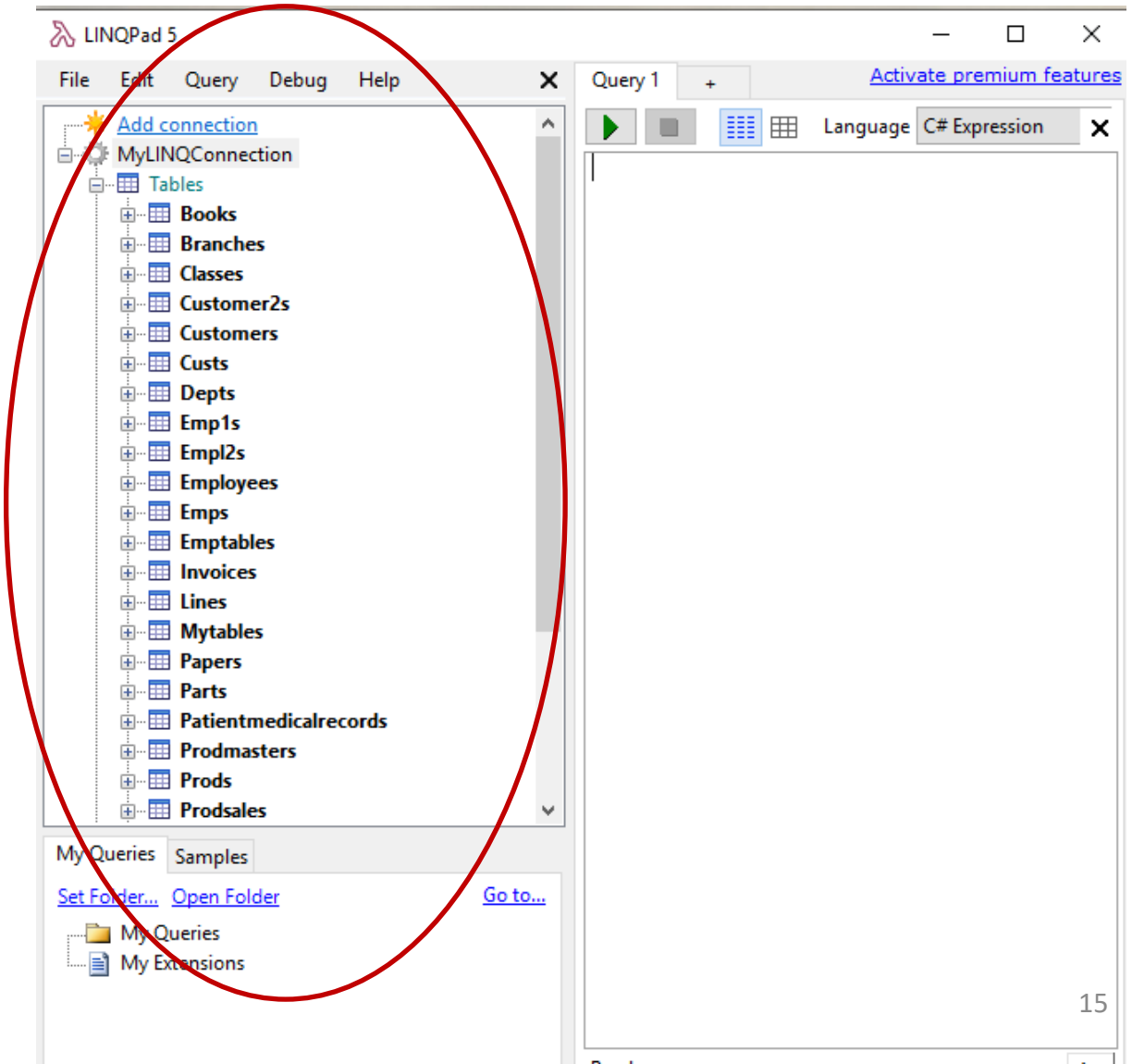
ConnectionName

Test



Connection made

- Now, you will see all your tables.
- Note the differences in naming conventions



LINQ Environment

We will be using C# syntax

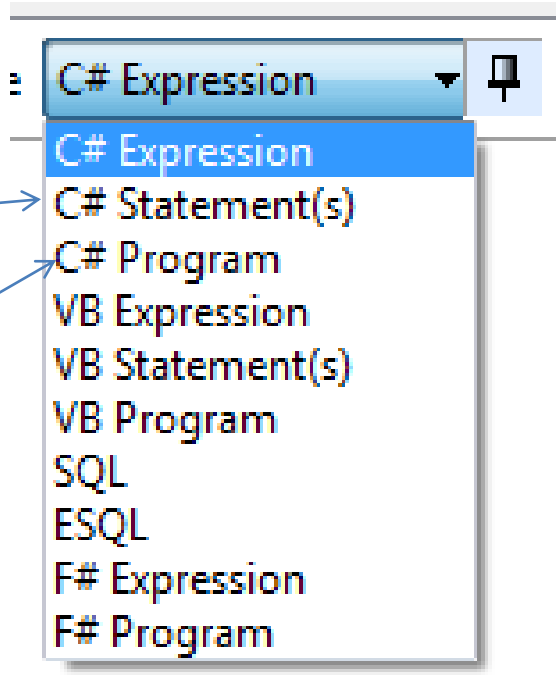
Also, later we will use C#

statements for writing queries

BUT, first

Let us write some simple C#

programs using the LINQ tool .



This will improve your understanding about the environment.

NOTE: Alternatively, you can write these simple programs in Visual Studio environment also by using the Console Application

LINQ Syntax

- Implicitly typed variable
- Query expression –

  from

  where

  select

- Filtering queries
- Grouping queries

NOTE: WE will use C# syntax

Implicitly typed variable

- Has a data type that is inferred from the expression used to initialize the variable
  - This process is called **type inference**
- To create an implicitly typed variable, you use a variable declaration with **`var`** as the data type

- Examples

```
var age = 22;  OR  int age = 22;

var name = " Jane"; OR  string name = "Jane";
```

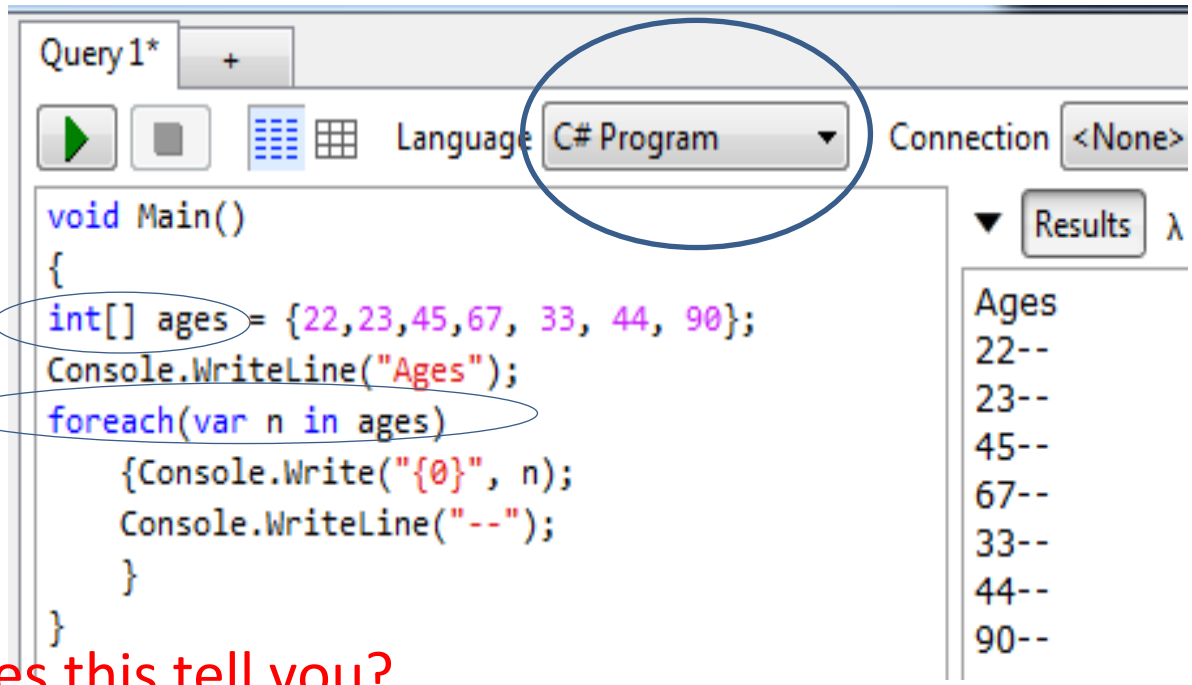
(http://msdn.microsoft.com/en-us/library/bb383973.aspx)

First we will use Arrays

- An array is a data structure that contains a number of variables called the elements of the array.
- All of the array elements must be of the same type
- Arrays are zero indexed

```
int[] myArray = {1, 3, 5, 7, 9};      implies myArray[0] = 1
string[] weekDays =
        {"Sun","Sat","Mon","Tue","Wed","Thu","Fri"};
                                  implies weekDays[6] = “Fri”
```

Implicitly typed variable



What does this tell you?

int[] – means an array of integers

ages – name of this array

var – means the datatype will be implicitly selected. i.e., it will be similar to the datatype elements defined in the array.

(i.e. var n will be of type int as is in the array ages)

What is (a) {0} in Console line? (b) difference between Write() and WriteLine()?

C# Programs

```
void Main()
{
int[] ages = {22,23,45,67, 33, 44, 90};
const int upperAgeLimit = 65;
Console.WriteLine("Ages below Age Limit");
foreach(var n in ages)
    {Console.Write("{0}", n < upperAgeLimit);
    Console.WriteLine();
    }
}
```

Results λ SQL IL Tree

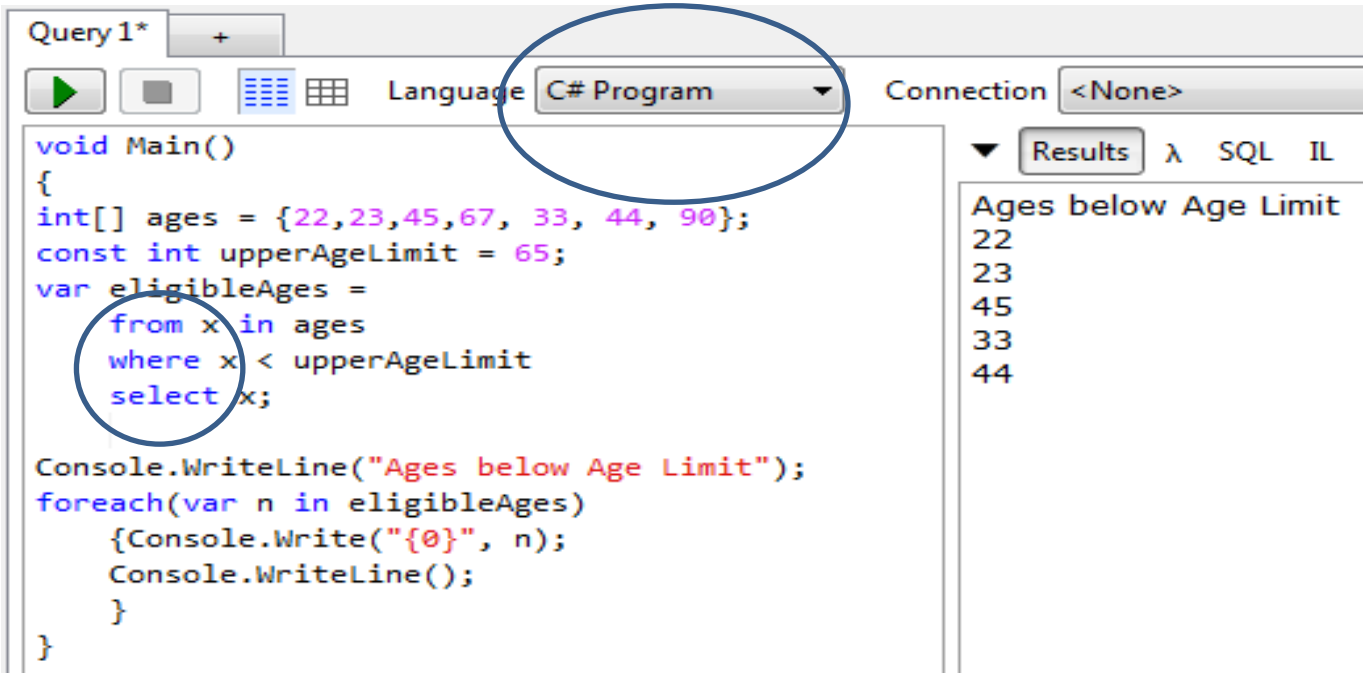
```
Ages below Age Limit
True
True
True
False
True
True
False
```

```
void Main()
{
int[] ages = {22,23,45,67, 33, 44, 90};
const int upperAgeLimit = 65;
Console.WriteLine("Ages below Age Limit");
foreach(var n in ages)
    {Console.Write("{0}", (n< upperAgeLimit) ? n :0) ;
    Console.WriteLine();
    }
}
```

Results λ SQL IL Tree

```
Ages below Age Limit
22
23
45
0
33
44
0
```

Implicitly typed variable



What does this tell you?

In this C# program, we have used a query expression – i.e. – Three clauses are used here ----- from, where and select

Another example with Objects

Language: C# Program Connection: <None>

```
void Main()
{
Student[] students = { new Student(1, "Andy", 5.5),
new Student(2, "Sam", 4.5),
new Student(3, "Tina", 3.5),
new Student(4, "Tom", 6.5),
new Student(5, "Xie", 7.5)};

const double BCutOff = 6.0;
var highGradeStudents =
    from x in students
    where x.GPA > BCutOff
    select x;

Console.WriteLine("Students with GPA > " + BCutOff);
Console.WriteLine();
foreach(var n in highGradeStudents)
    {Console.Write("{0, 3} {1, 5} {2,4}", n.ID,
    n.Name, n.GPA);
    Console.WriteLine();
    }
}
class Student
{
public Student(int num, string name, double gpa)
    {
    ID = num;
    Name = name;
    GPA = gpa;
    }
    public int ID { get; set;}
    public string Name {get; set;}
    public double GPA { get; set;}
}
```

Results | λ | SQL | IL | Tree

```
Students with GPA > 6

  4   Tom  6.5
  5   Xie  7.5
```

What does this tell you?

In this C# program, we have used a

1. Class Student
2. 5 objects

and

3. A query expression

Note the use of

```
{Console.Write("{0, 3} {1, 5} {2,4}", n.ID, n.Name, n.GPA);
```

Interface -- IEnumerable

The IEnumerable interface is used to iterate through a collection. The results are dynamically created as you start to iterate through the collection via the foreach statement.

IEnumerable<string> - Here we have typecasted the collection as a collection of strings.

Similarly, we can typecast to other datatypes

IEnumerable<int>

Example of IEnumerable

```
IEnumerable <string> Members =  get an array/collection
                             = from nm in names
                               select x;
```

Now, just iterate thru the collection

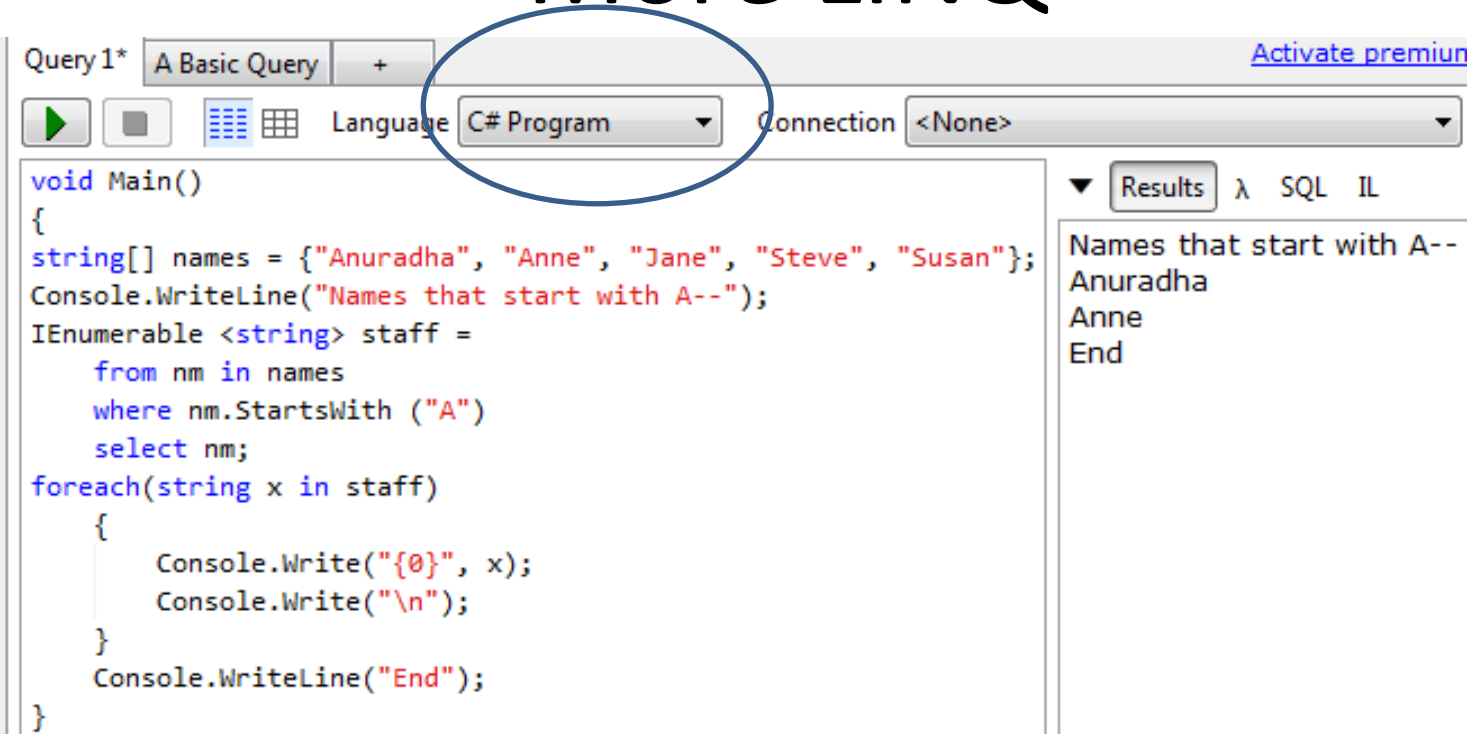
```
foreach (string value in Members)
        { Console.WriteLine(value); }
```

OR

```
foreach (string value in Members)
                { Console.Write(value);
                Console.Write("\n"); }
```

The IEnumerable<T> interface is a generic interface that provides an abstraction for looping over elements.

More LINQ



What does this tell you?

In this C# program, an array of string datatypes (called names) is filtered, to give only those names which start with "A"

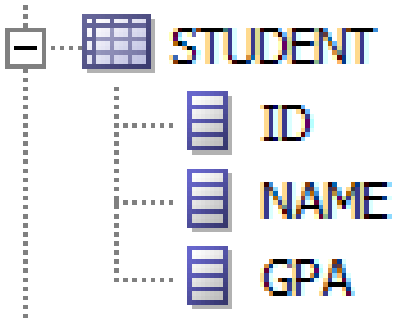
Note: Console.Write("\n"); means next line

Next, using databases

Using SQL Developer – first, create a table called STUDENT

is337002Conn

```
1 drop table student;
2 create table Student(
3 ID number(2) primary key,
4 Name varchar2(30),
5 GPA number (3,1))
```



Write SQL to insert data

is123456Conn STUDENT 0.5307548 seconds

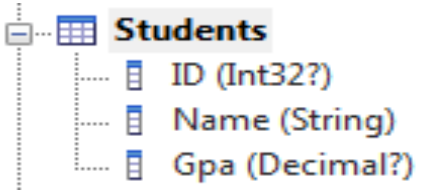
```
1 INSERT INTO Student VALUES(1, 'AAA', 5.5);
2 INSERT INTO Student VALUES(2, 'BBB', 4.5);
3 INSERT INTO Student VALUES(3, 'CCC', 3.5);
4 INSERT INTO Student VALUES(4, 'DDD', 6.5);
5 INSERT INTO Student VALUES(5, 'EEE', 7.5);
```

	ID	NAME	GPA
1	1	AAA	5.5
2	2	BBB	4.5
3	3	CCC	3.5
4	4	DDD	6.5
5	5	EEE	7.5

Commit;

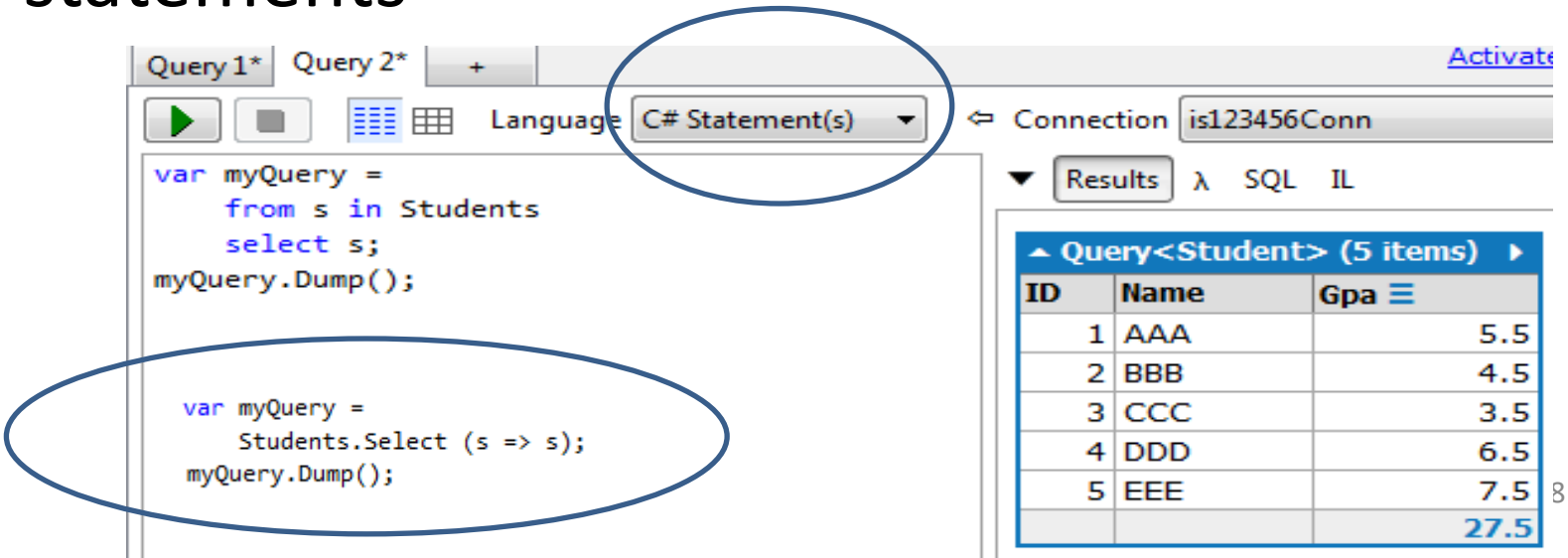
Next - Using LINQ

- First make a connection to Oracle database



Note the table is named as Students (not Student). This is because, we will be treating this as an array or a collection, hence the plural.

- Writing simple query statements using LINQ or C# statements



More LINQ

Query 1* | Query 2* | + | Activate

Language: C# Statement(s) | Connection: is123456Conn

```
const decimal BCutOff = 6.0M;
var myQuery =
    from s in Students
    where s.Gpa > BCutOff
    select s;
myQuery.Dump();
```

Results | λ | SQL | IL

Query<Student> (2 items)

ID	Name	Gpa
4	DDD	6.5
5	EEE	7.5
		14.0

Or

```
var myQuery =
from s in Students
where s.Gpa > 6.0M
select s;
myQuery.Dump();
```

Is same as the SQL statement

is123456Conn | STUDENT | 0.00452533 secon

```
1 select * from student where gpa > 6.0;
```

Results | Script Output | Explain | Autotrace

Results:

	ID	NAME	GPA
1	4	DDD	6.5
2	5	EEE	7.5

More LINQ...

LINQ queries are in

(a) Query syntax →

```
var myQuery =
from s in Students
where s.Gpa > 6.0M
select s;
myQuery.Dump();
```

(b) Lambda syntax →

```
var myQuery =
Students.Where(s=>s.Gpa>6.0M);
myQuery.Dump();
```

In this paper, we will cover Query syntax, since they are more close to SQL, and so are easier to relate to all that we have learnt earlier in this paper.

Lambda syntax on the other hand is closer to writing code in the VB/C# programming environment, and used by geeks.



Assigning Values to Constants

- Declare the data type of numeric constants by appending a type-declaration character

	VB	C#	VB		C#
Decimal	**D**	**M**	**850.50D**	**or**	**850.50M**
Double	**R**	**D**	**52875.8R**	**or**	**52875.8D**
Integer	**I**	**I**	**12345678I**		
Long	**L**	**L**	**134257987L**		
Single/ Float	**F**	**F**	**101.25F**		

When declaring a constant in Visual Studio languages (VB/C#)

If a type-declaration character is not appended as shown above, then any whole number is assumed to be Integer and any fractional value is assumed to be Double

e.g. VB – Const BCutOff as Decimal = 6.0D

C# – const decimal BCutOff =6.0M;

Vendor, Product, Lines – Tables

```
var myQuery =
    from v in Vendors
    select v;
myQuery.Dump();
```

Query<Vendor> (12 items)

VCode	VName	VContact	VAreacode	VPhone	VState	VOrder	Products
21225	Bryson, Inc.	Smithson	615	223-3234	TN	Y	Products
21226	SuperLoo, Inc.	Flushing	904	215-8995	FL	N	Products
21231	DeeEee Supply	Singh	615	228-3245	TN	Y	Products
21344	Gomez Bros.	Ortega	615	889-2546	KY	N	Products

Products

EntitySet<Product> (2 items)

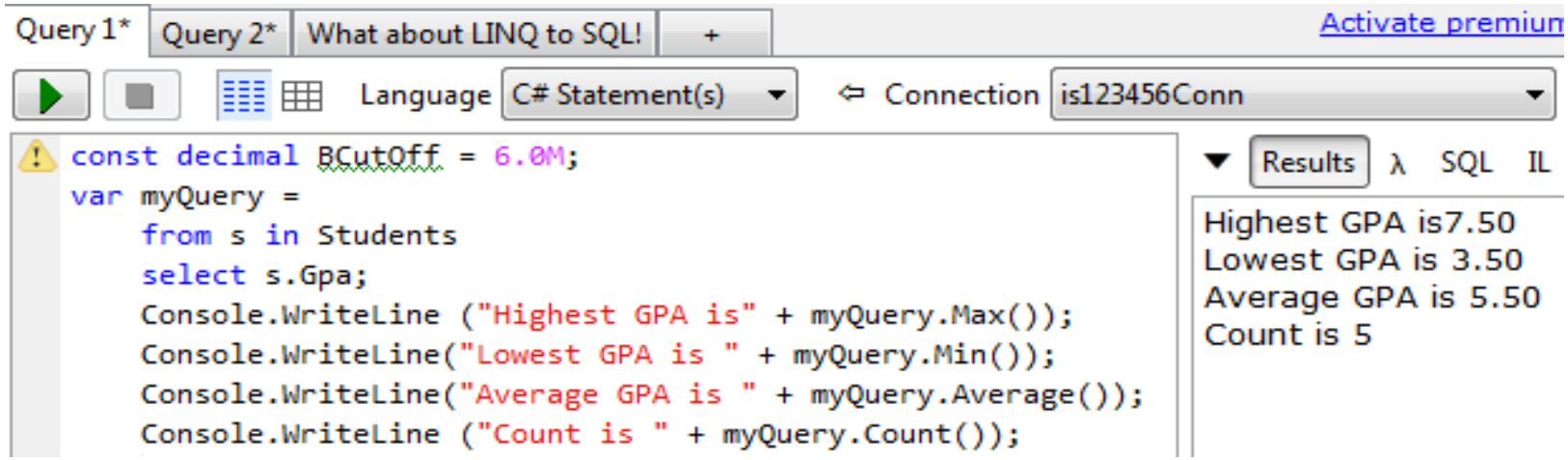
PCode	PDescript	PIndate	PQoh	PMin	PPrice	PDiscount	VCode	PReorder	Lines
23109-HB	Claw hammer	20/01/2012 12:00:00 AM	23	10	9.95	0.1	21225	0	Lines
SM-18277	1.25-in. metal screw, 25	1/03/2012 12:00:00 AM	172	75	6.99	0	21225	0	Lines
			195	85	16.94	0.1	42450	0	

EntitySet<Line> (4 items)

InvNumber	LineNumber	PCode	LineUnits	LinePrice
1001	2	23109-HB	1	9.95
1004	2	23109-HB	2	9.95
1006	3	23109-HB	1	9.95
1008	3	23109-HB	1	9.95
4019	10		5	39.80

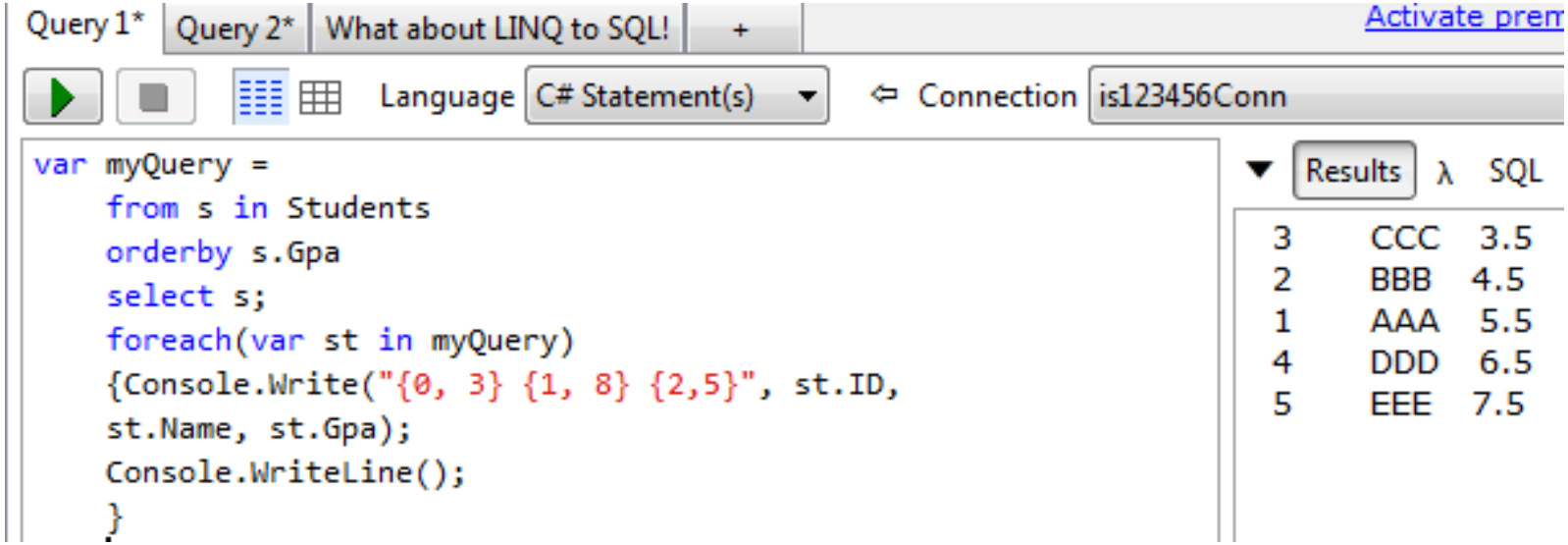
LINQ Queries

- You can use several **aggregate operators**
  - Operators that produce statistics for groups of data
  - Include `Average()`, `Count()`, `Sum()`, `Max()`, and `Min()`

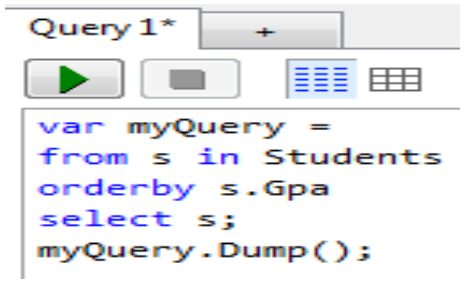


LINQ Operators to Sort Data

- **`orderby` operator**
  - Used to sort a collection of data based on a field or fields



Or



Insert a New Student Record

Query 1* +

Language C# Statement(s)

```
Student std = new Student
{
    ID = 12,
    Name = "YYY",
    Gpa = 3.7M
    };

// Add the new object to the Students collection.
Students.InsertOnSubmit(std);
// Submit the change to the database.
try
{
    SubmitChanges();
}
catch (Exception e)
{
    Console.WriteLine(e);

}
```

is337002Conn | STUDENT

Columns | Data | Constraints | Grants | Sta

Sort... | Fil

	ID	NAME	GPA
1	1	AAA	5.5
2	2	BBB	4.5
3	3	CCC	3.5
4	4	DDD	6.5
5	5	EEE	7.5
6	11	XXX	3.7
7	12	YYY	3.7

Delete a Student

Single Student

Query 1* + Activate premium f

Language C# Statement(s) ⇦ Connec

```
Student std = Students.Where(s => s.ID == 11).Single();
// Delete the object to the Students collection.
Students.DeleteOnSubmit(std);
// Submit the change to the database.
try
{
    SubmitChanges();
}
catch (Exception e)
{
    Console.WriteLine(e);

}
```

Note the use of Single() here

is337002Conn STUDENT

Columns Data Constraints Grants

Sort...

	ID	NAME	GPA
1	1	AAA	5.5
2	2	BBB	4.5
3	3	CCC	3.5
4	4	DDD	6.5
5	5	EEE	7.5
6	12	YYY	3.7

Delete a Group of Students

Students with Gpa< 4

Query 1* + Activate premium feat

Language C# Statement(s) Connection

```
IEnumerable <Student> stds = Students.Where(s => s.Gpa < 4);
// Delete the object to the Students collection.
foreach (var std in stds)
{
Students.DeleteOnSubmit(std);
}
try
{
    SubmitChanges();
}
catch (Exception e)
{
    Console.WriteLine(e);
}
```

Note the use of IEnumerable here, as we have to loop thru the collection

is337002Conn STUDENT

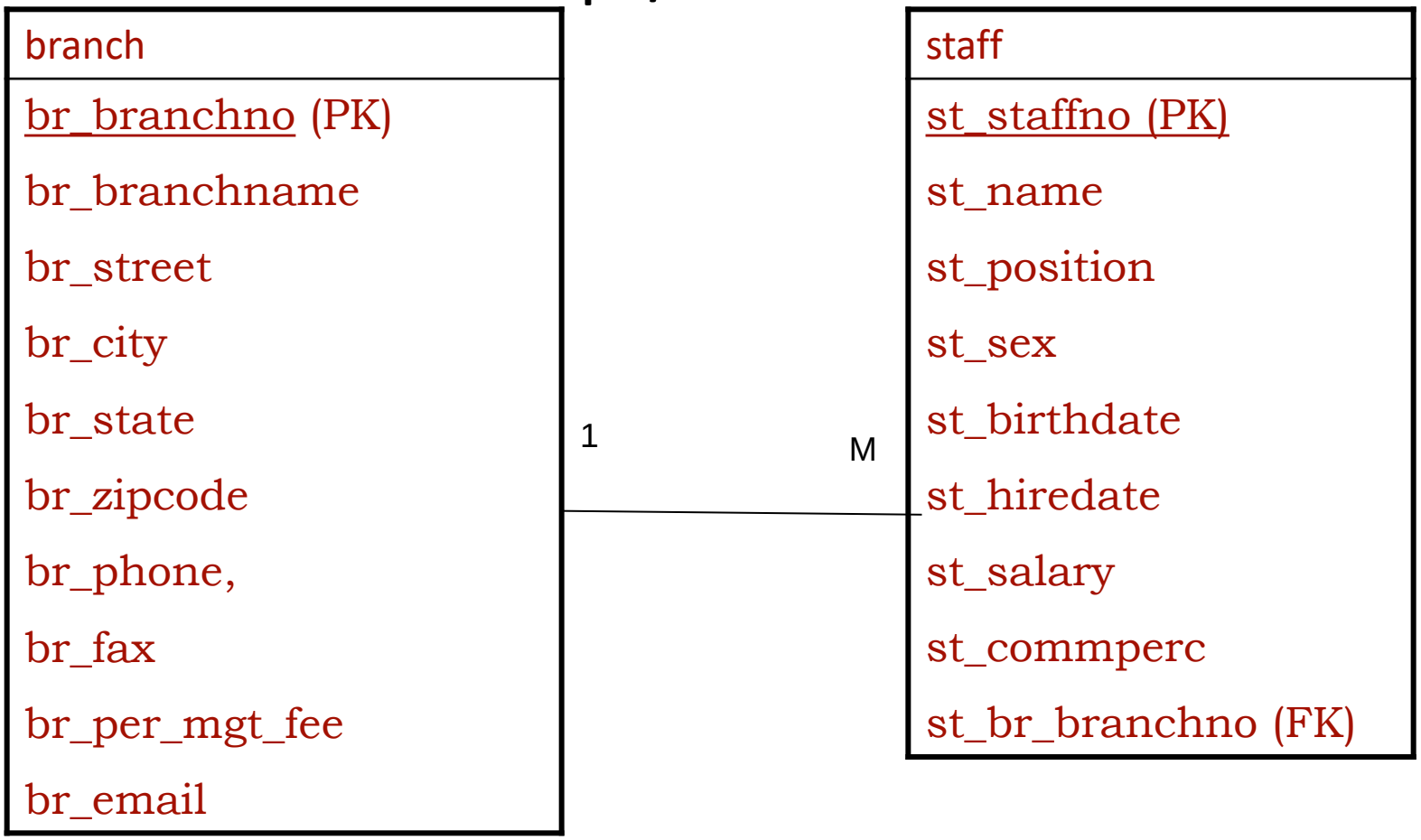
Columns Data Constraints Grants Stat

Sort... Filt

	ID	NAME	GPA
1	1	AAA	5.5
2	2	BBB	4.5
3	4	DDD	6.5
4	5	EEE	7.5

Associations or Joins

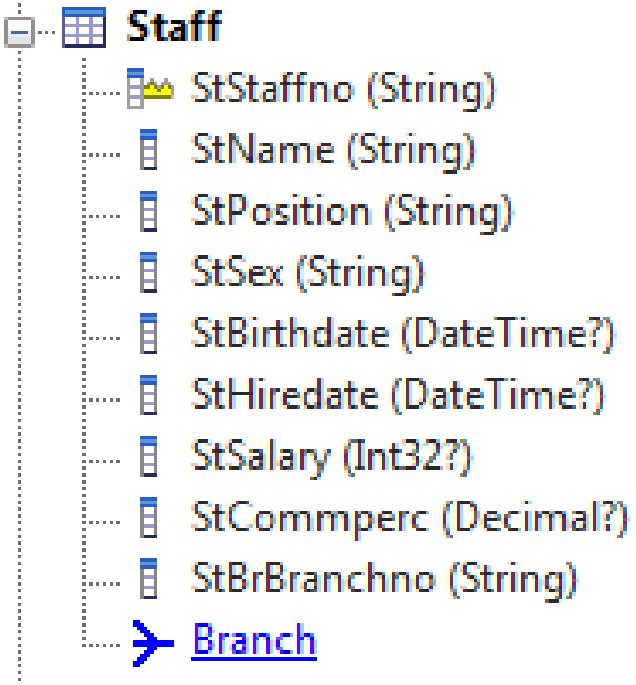
Another benefit of LINQ is that you can query across relationships/associations.

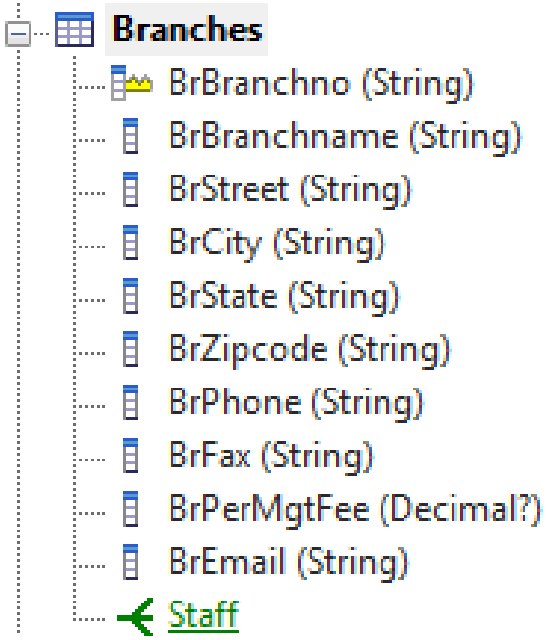


Associations or Joins

A Staff member belongs to one Branch only

A Branch can have many staff members





Joins

Clauses used are – from, join, on, where & select

from t1 in Table1 join t2 in Table2

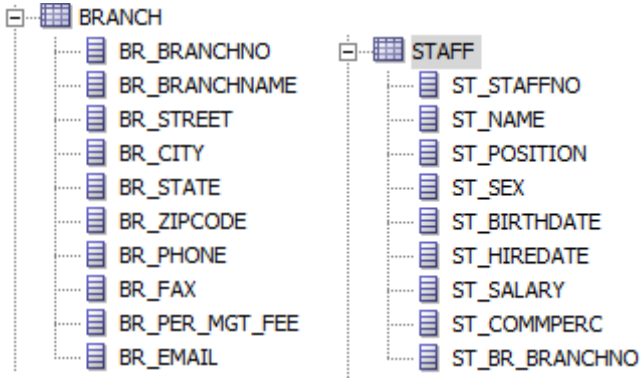
On t1.attributeX equals t2.attribute Y

```
var myQuery =
from b in Branches join s in Staff
on b.BrBranchno equals s.StBrBranchno
select new
{ BranchNumber = b.BrBranchno,
BranchName = b.BrBranchname,
StaffName = s.StName
};
myQuery.Dump();
```

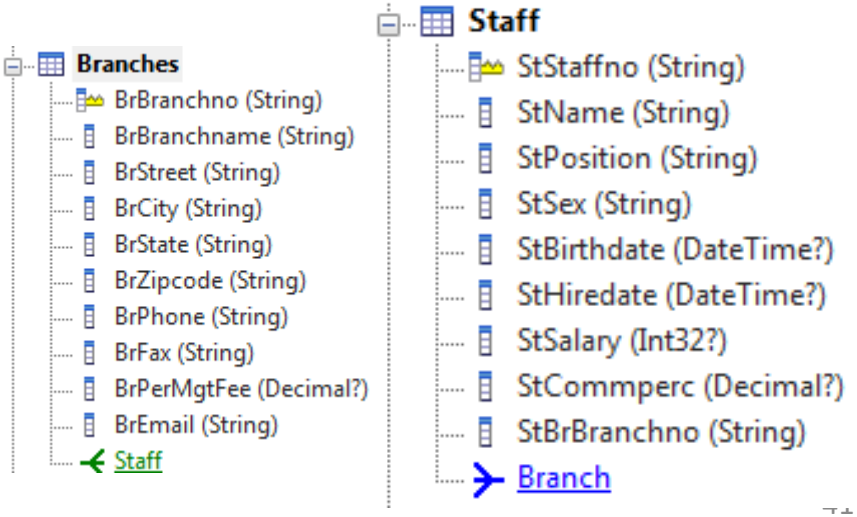
SQL and LINQ

List the names and positions of staff members who work in the Brenham branch. Assume that the branch number of the Brenham branch is not known (i.e. more than one table needs to be referenced in this query).

```
select staff.* from staff, branch
where staff.st_br_branchno = branch.br_bran
and branch.br_branchname = 'Brenham'
```




```
string Location = "Brenham";
var myQuery =
from b in Branches join s in Staff
on b.BrBranchno equals s.StBrBranchno
where b.BrBranchname == Location
select new
{
BranchNo = b.BrBranchno,
BranchName = b.BrBranchname,
StaffNo = s.StStaffno,
StaffName = s.StName
} ;
myQuery.Dump();
```



SQL and LINQ

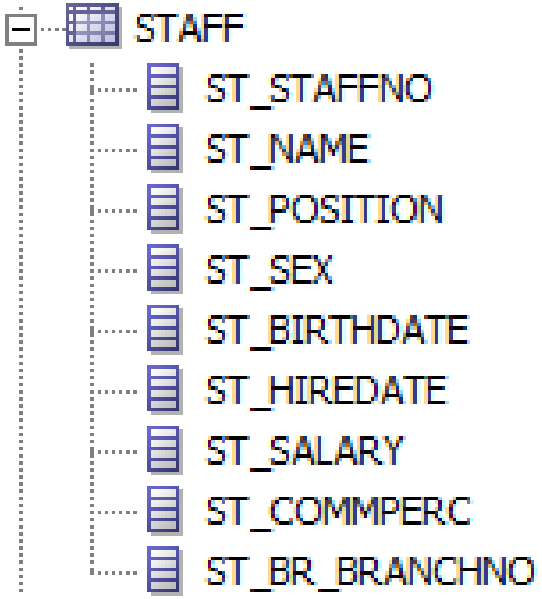
Display the names of branches with staff having supervisor positions.

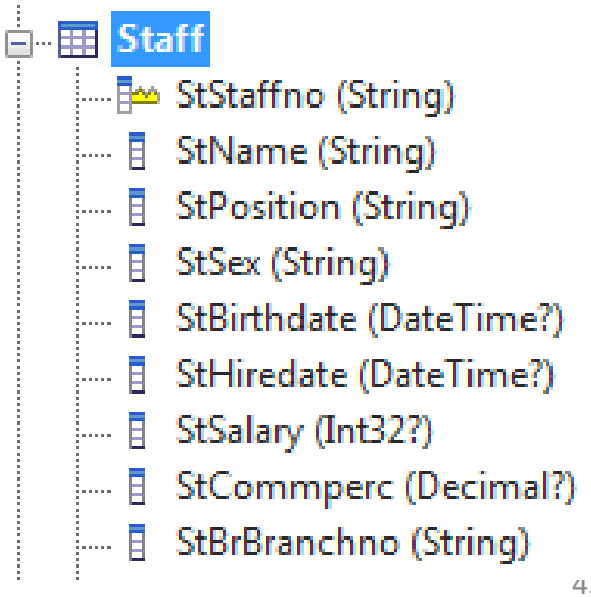
```
string Position = "Supervisor";
var myQuery =
from b in Branches join s in Staff
on b.BrBranchno equals s.StBrBranchno
where s.StPosition == Position
select new
{
BranchNo = b.BrBranchno,
BranchName = b.BrBranchname
} ;
myQuery.Dump();
```

```
1 select st_br_branchno, br_branchname
2 from staff, branch
3 where st_position = 'Supervisor'
4 and st_br_branchno = br_branchno
```

LINQ Operators to Group Data

- **`group` operator**
  - Groups data by specified criteria
  - The number of possible values in the criterion determines the number of possible groups
  - Staff





Group by operator

Display the total number of staff in each branch

```
var myQuery =
from b in Branches join s in Staff
on b.BrBranchno equals s.StBrBranchno
group s.StBrBranchno by b.BrBranchname into g
select new
{
BranchNo = g.Key,
TotalStaff = g.Count()
} ;
myQuery.Dump();
```

Query<> (8 items)

BranchNo	TotalStaff
Memorial	3
Inner Loop	2
Northwest	2
The Woodlands	2
Round Top	1
Fort Bend	2
Brenham	3
West	2
	17

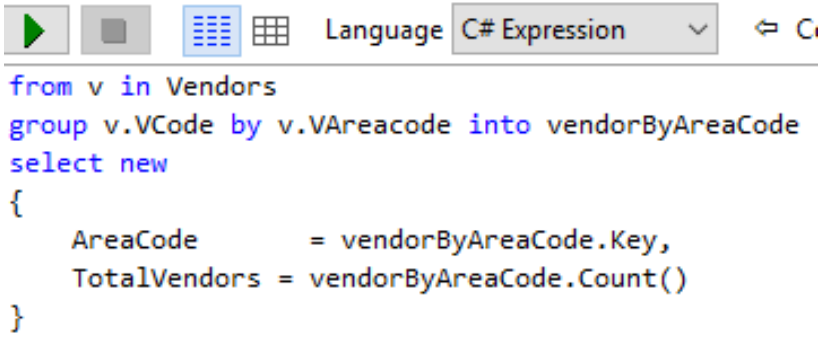
```
1 select br_branchname, count(st_staffno) "Number of Staff"
2 from staff, branch
3 where st_br_branchno = br_branchno
4 group by br_branchname
```

```
BR_BRANCHNAME        Number of Staff
-------------------- ----------------
Memorial             3
Inner Loop           2
Northwest            2
The Woodlands        2
Round Top            1
Fort Bend            2
Brenham              3
West                 2

8 rows selected
```

Grouping Algorithm

```
from t1 in Table1
group t1.attribute1 by  t1.attribute2 into groupname
select new
{
Year       = groupname.Key,
TotalValue = groupname.Count()
}
```

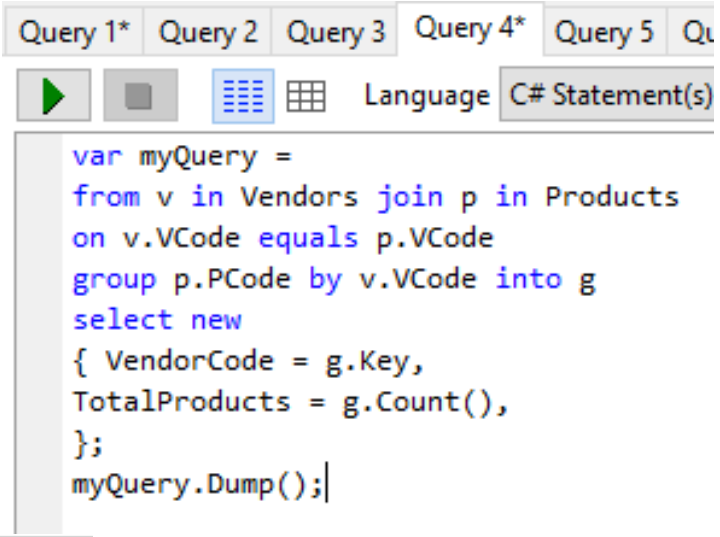


Query<> (4 items)

AreaCode	TotalVendors
904	3
222	1
615	6
901	2
	12

Grouping Algorithm

```
var myQuery =
from t1 in Table1 join t2 in Table2
on t1.attributeA equals t2.attributeA
group t1.attributeA by t2.attributeB into g
select new
{
ColumnName = g.Key,
SummaryInfo = g.Count()
}
myQuery.Dump();.
```



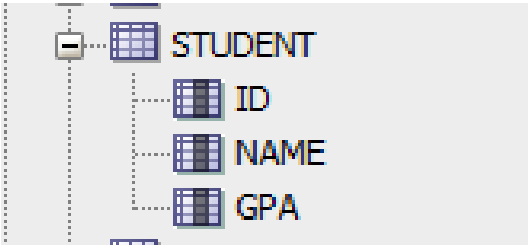
Query<> (6 items)

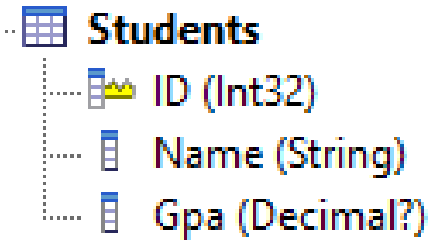
VendorCode	TotalProducts
25595	3
23119	2
21231	1
21225	2
24288	3
21344	3
136802	14

Non Subversion Rule

Non-Subversion	If the system supports low-level access to the data, users must not be allowed to bypass the integrity rules of the database.

Add a trigger (TRG_STUD_ID_ADD)which adds ID to the Student Table






```
Student std = new Student
{
    Name = "ABCDEF",
    Gpa = 7.7M
    };

// Add the new object to the Students collection
Students.InsertOnSubmit(std);
// Submit the change to the database.
try
{
    SubmitChanges();
}
catch (Exception e)
{
    Console.WriteLine(e);

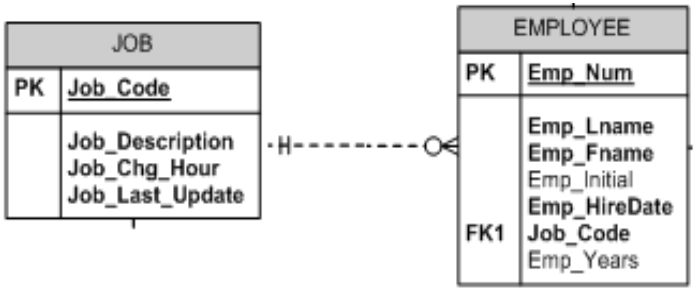
}
```

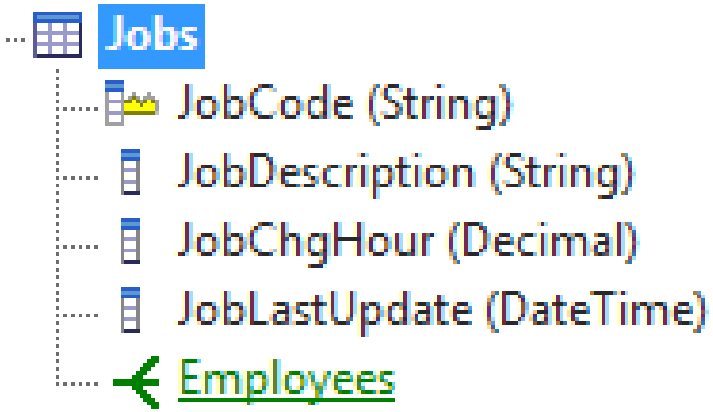
Query<Student> (8 items)

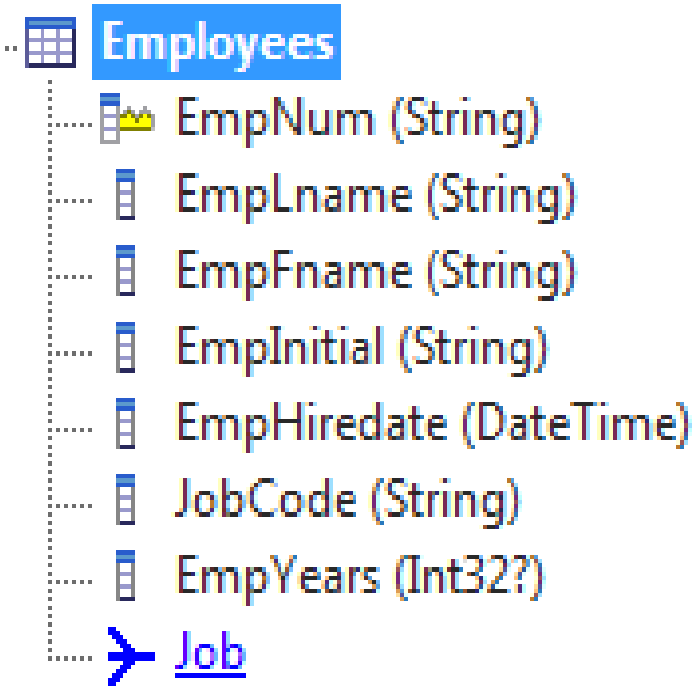
ID	Name	Gpa
12	YYY	3.7
13	xxx	3.7
1	AAA	5.5
2	BBB	4.5
3	CCC	3.5
4	DDD	6.5
5	EEE	7.5
14	ABCDEF	7.7

Tutorial Exercise – Week 9

Simple Problem Domain







LINQ examples

- Many LINQ resources are provided in LINQPad. Suggest you go over them whenever you have some spare time.
- Languages – C#, F# & VB
- Databases – Oracle, MySQL, SQLite, etc..
- Code snippets

Using Visual Studio

```
namespace ConsoleApp1
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            int[] ages = { 22, 23, 45, 67, 33, 44, 90 };
            Console.WriteLine("Ages");
            foreach (var n in ages)
            {
                Console.Write("{0}", n);
                Console.WriteLine("--");
            }
            Console.ReadLine();
        }
    }
}
```

```
class Program
{
    1 reference
    class Student
    {
        0 references
        public Student(int num, string name, double gpa)
        {
            ID = num;
            Name = name;
            GPA = gpa;
        }
        1 reference
        public int ID { get; set; }
        1 reference
        public string Name { get; set; }
        1 reference
        public double GPA { get; set; }
    }

    0 references
    static void Main(string[] args)
    {

        string[] names = {"Anuradha", "Anne", "Jane", "Steve", "Susan"};

        Console.WriteLine("Names that start with A----");
        IEnumerable <string> staff =
        from nm in names
        where nm.StartsWith ("A")
        select nm;

        foreach(string x in staff)
            {Console.Write("{0}", x);
            Console.WriteLine();
            }
         Console.WriteLine("End of Prog");
         Console.ReadLine();
        }
                }
}
```

Using Visual Studio

Add references to relevant Oracle Libraries

```
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using Oracle.ManagedDataAccess;
using Oracle.ManagedDataAccess.Client;
using System.Data.Linq;
using System.Data.Linq.Mapping;
```

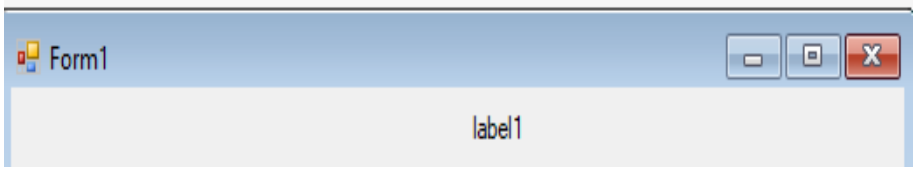
```
namespace WindowsFormsApp1
{
    3 references
    public partial class Form1 : Form
    {
        1 reference
        public Form1()...

        string constr, cmdSQL;
        1 reference
        private void Form1_Load(object sender, EventArgs e)
        {
            // Connect to Oracle
            constr = "User Id=it123456;Password=[redacted];Data Source=inms-oracle.massey.ac.nz:1521/orcl.massey.ac.nz";
            OracleConnection con = new OracleConnection(constr);
            con.Open();

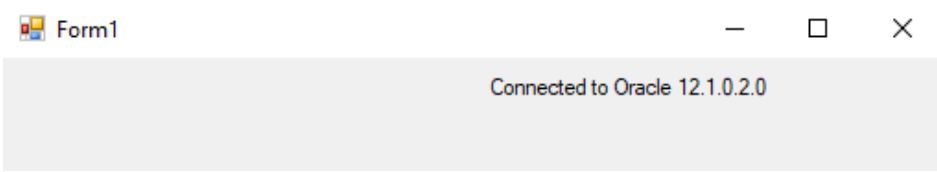
            // Display Version Number
            label1.Text = "Connected to Oracle " + con.ServerVersion;

            // Close and Dispose OracleConnection
            con.Close();
            con.Dispose();
        }
```

Compile Time

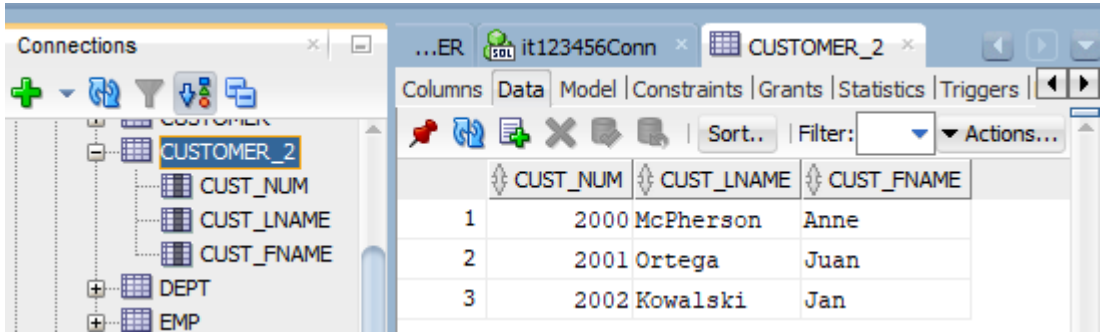


Run Time



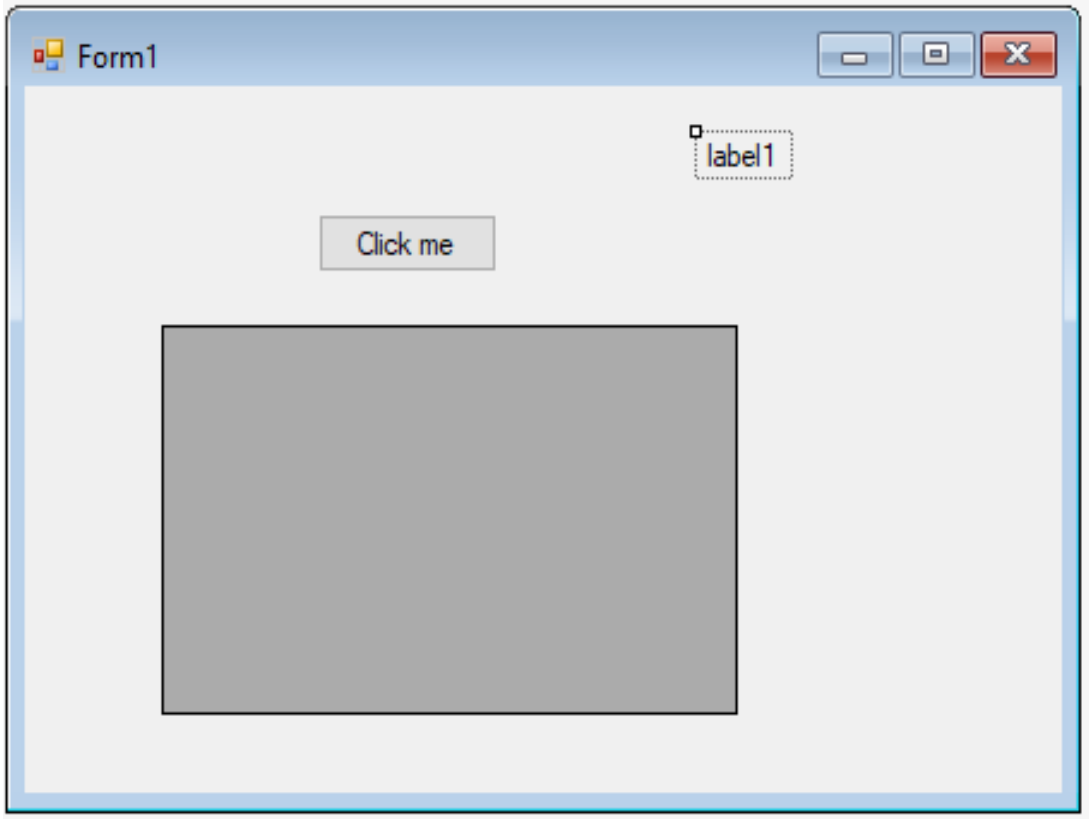
Using Visual Studio to Access Data

SQLDeveloper View of Table CUSTOMER_2



Add two more controls –

- Button
- DataGridView



Visual Studio, SQL & Oracle

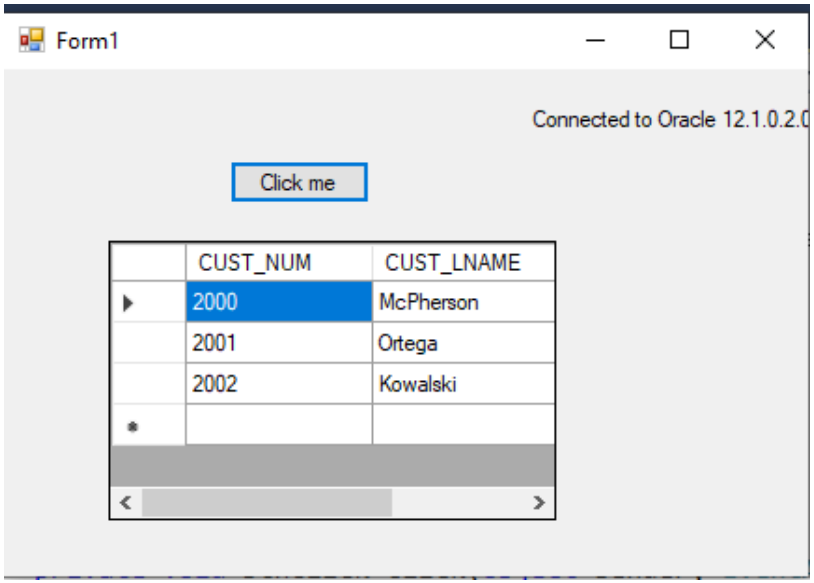
```
1 reference
private void btnClick_Click(object sender, EventArgs e)
{
    cmdSQL = "select cust_num, cust_Lname, cust_fname from customer_2";

    // connection string
    OracleDataAdapter adapter = new OracleDataAdapter(cmdSQL, constr);

    // Create the builder for the adapter to automatically generate the Command when needed
    OracleCommandBuilder builder = new OracleCommandBuilder(adapter);

    // Create and fill the DataSet using the CUSTOMER_2
    DataSet dataset = new DataSet();
    adapter.Fill(dataset, "CUSTOMER_2");

    // Get the CUSTOMER_2 table from the dataset
    DataTable table = dataset.Tables["CUSTOMER_2"];
    // fill datagrid view
    dataGridViewCust.DataSource = table;
}
```

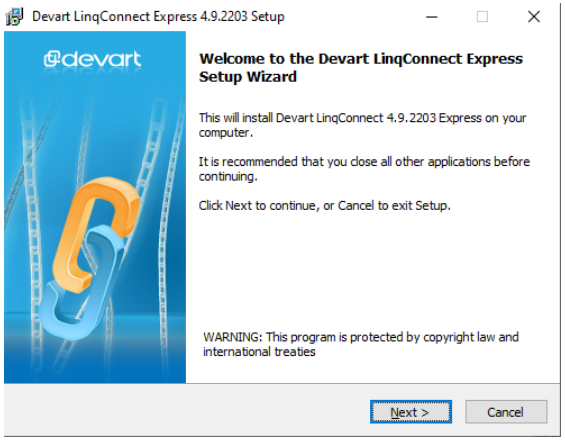


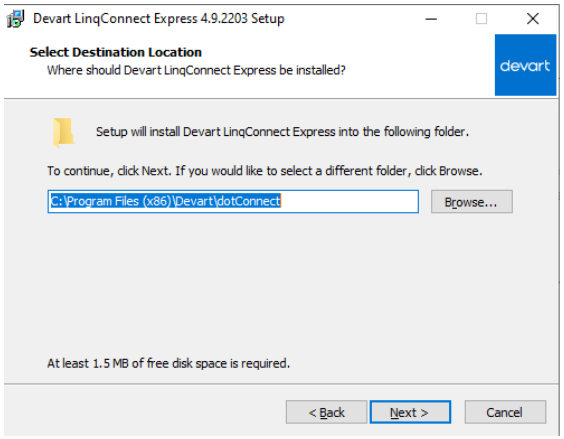
Visual Studio, LINQ & Oracle

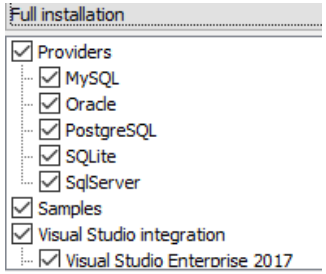
- We cannot access Oracle database directly with LINQ.
- For accessing the Oracle database, you will have to use the **Entity Framework**
- The Linq-to-SQL is a **SQL Server** only solution - it was designed by the C# team (not the database folks) as a "proof-of-concept" for the LINQ capabilities.
- We will use the free LinqConnect

LinqConnect 4.9.2203 Express

- Download from https://marketplace.visualstudio.com/items?itemName=DevartSoftware.LinqConnectExpress-LINQtoSQLSolutionforanyDatabase
- Install



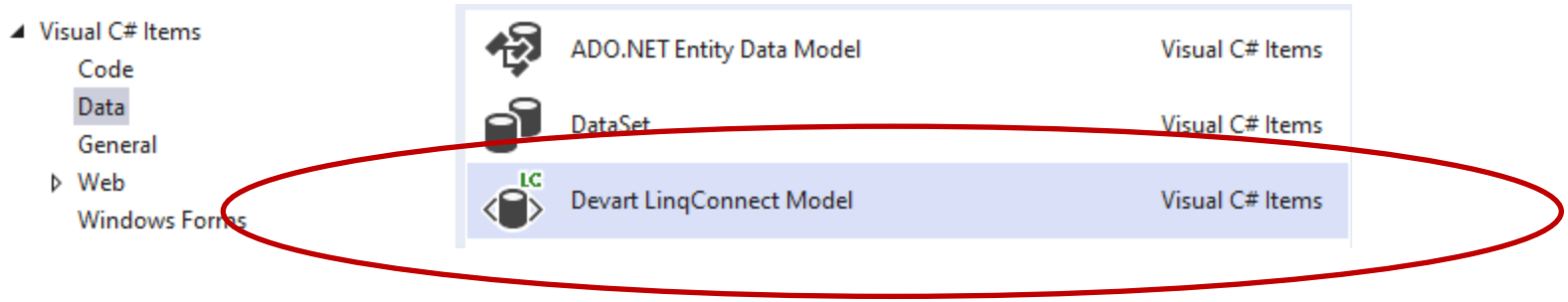




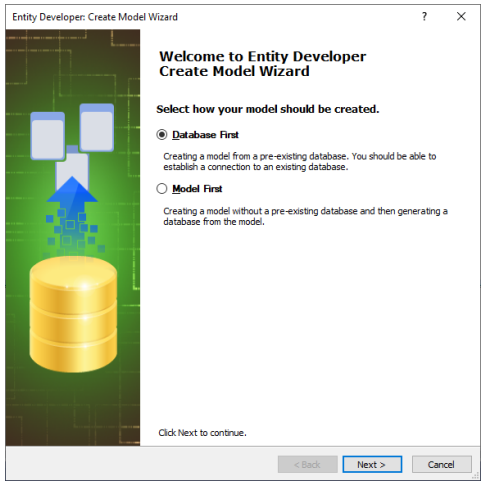
Visual Studio 2019 Community

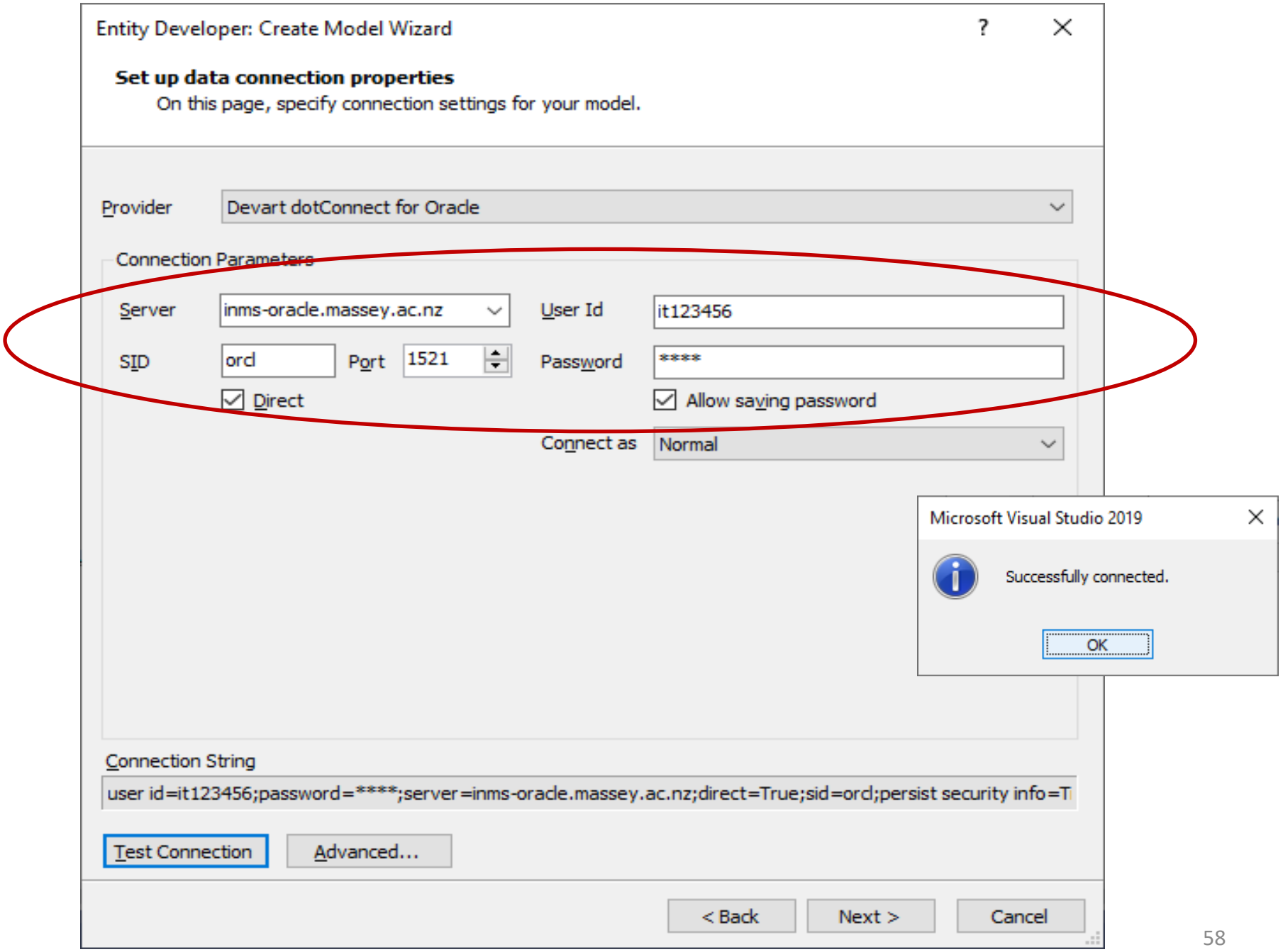
Refer - https://www.devart.com/dotconnect/oracle/articles/tutorial_linq.html

Add new item >> Devart LinqConnect Model

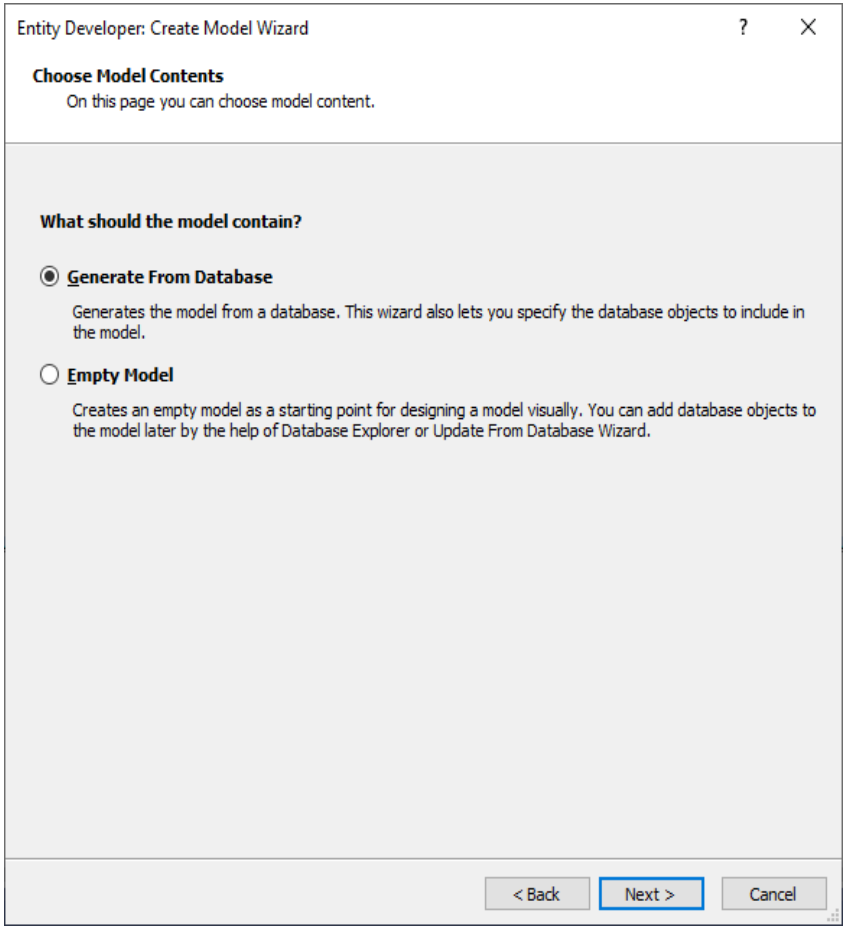


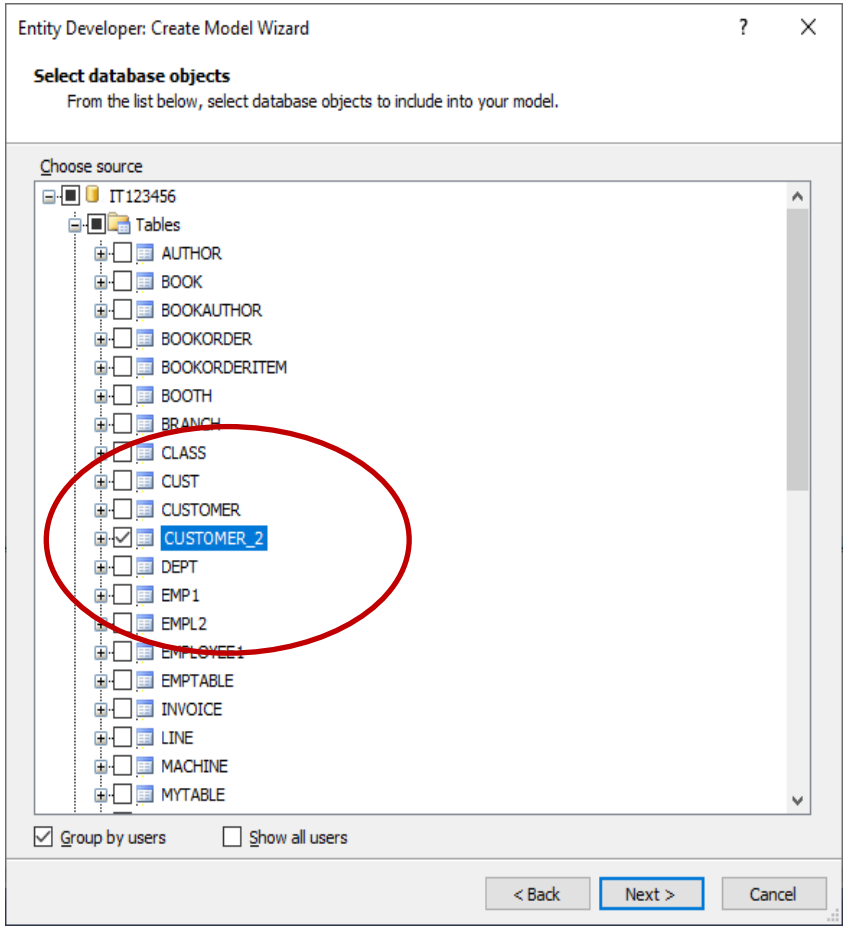
Wizard

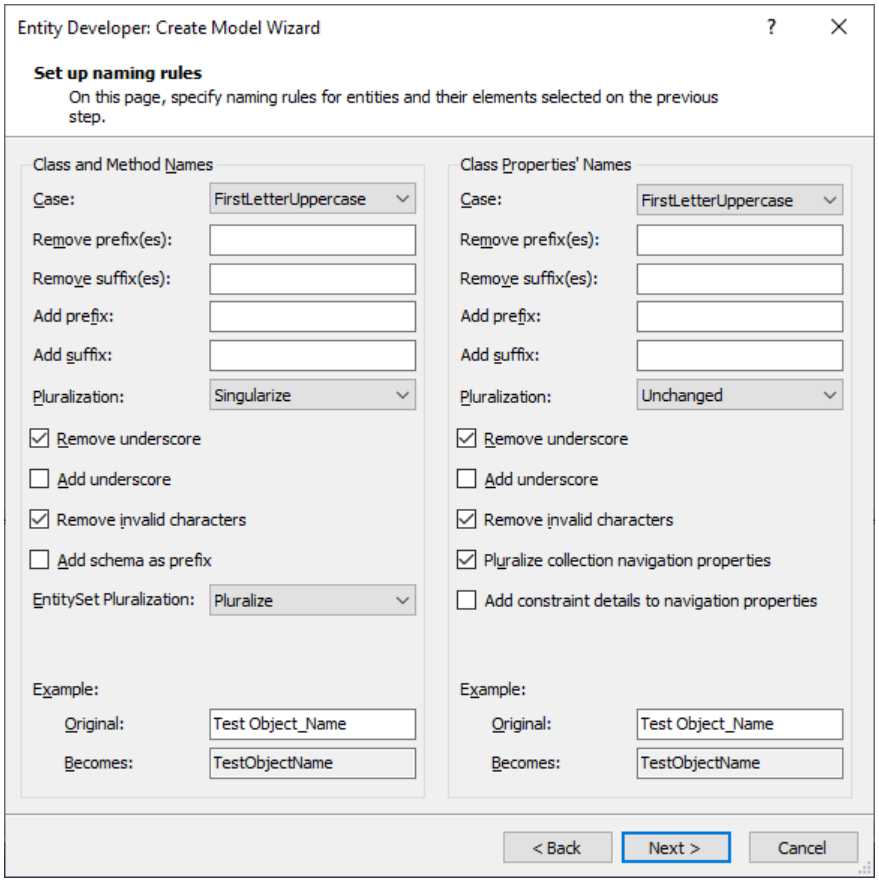


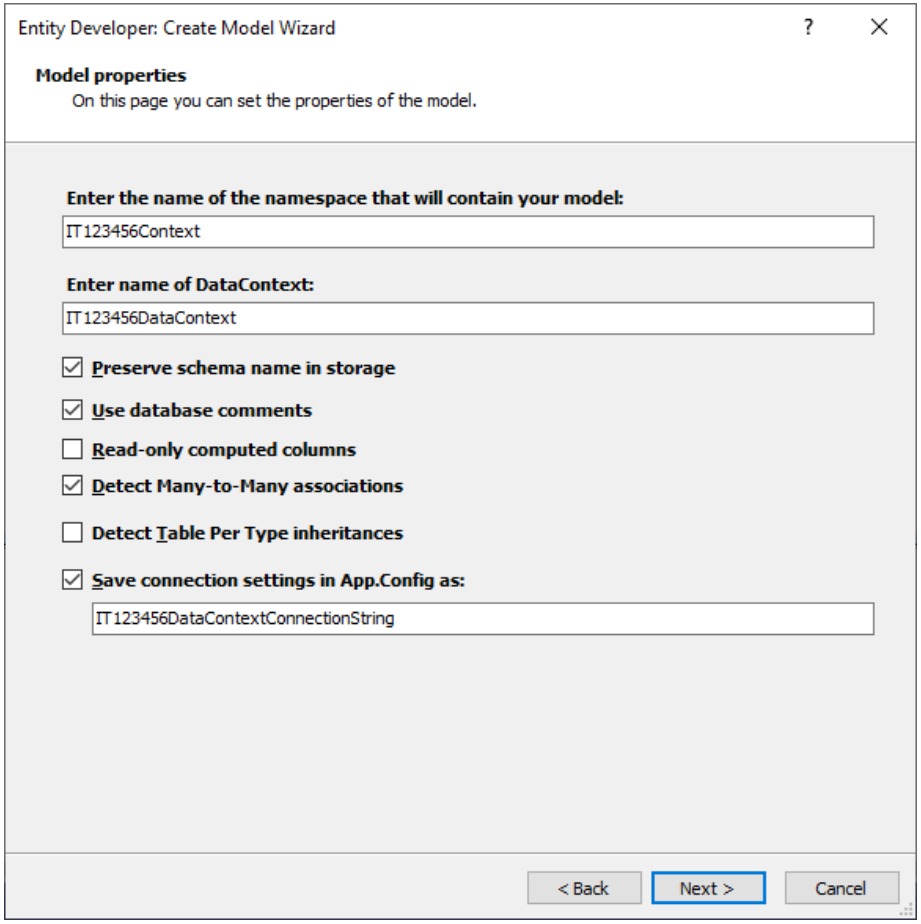

Entity Developer: Create Model Wizard
Set up data connection properties
On this page, specify connection settings for your model.
Provider
Devart dotConnect for Oracle
Connection Parameters
Server
inms-oracle.massey.ac.nz
User Id
it123456
SID
orcl
Port
1521
Password
****
Direct
Allow saving password
Connect as
Normal
Microsoft Visual Studio 2019
Successfully connected.
OK
Connection String
user id=it123456;password=****;server=inms-oracle.massey.ac.nz;direct=True;sid=orcl;persist security info=T
Test Connection
Advanced...
< Back
Next >
Cancel

Generate from Database

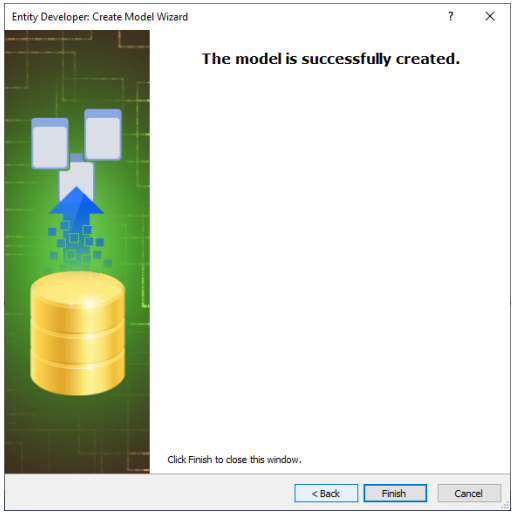




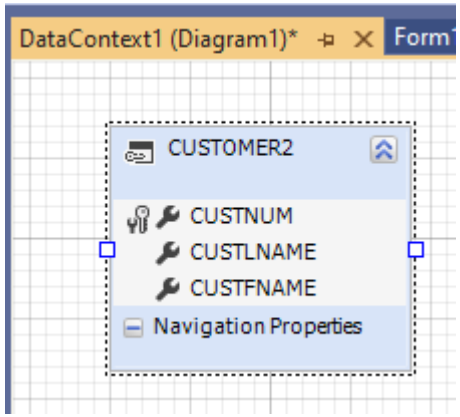

Entity Developer: Create Model Wizard
Set up naming rules
On this page, specify naming rules for entities and their elements selected on the previous step.
Class and Method Names
Case: FirstLetterUppercase
Remove prefix(es):
Remove suffix(es):
Add prefix:
Add suffix:
Pluralization: Singularize
Remove underscore
Add underscore
Remove invalid characters
Add schema as prefix
EntitySet Pluralization: Pluralize
Example:
Original: Test Object_Name
Becomes: TestObjectName
Class Properties' Names
Case: FirstLetterUppercase
Remove prefix(es):
Remove suffix(es):
Add prefix:
Add suffix:
Pluralization: Unchanged
Remove underscore
Add underscore
Remove invalid characters
Pluralize collection navigation properties
Add constraint details to navigation properties
Example:
Original: Test Object_Name
Becomes: TestObjectName
< Back
Next >
Cancel

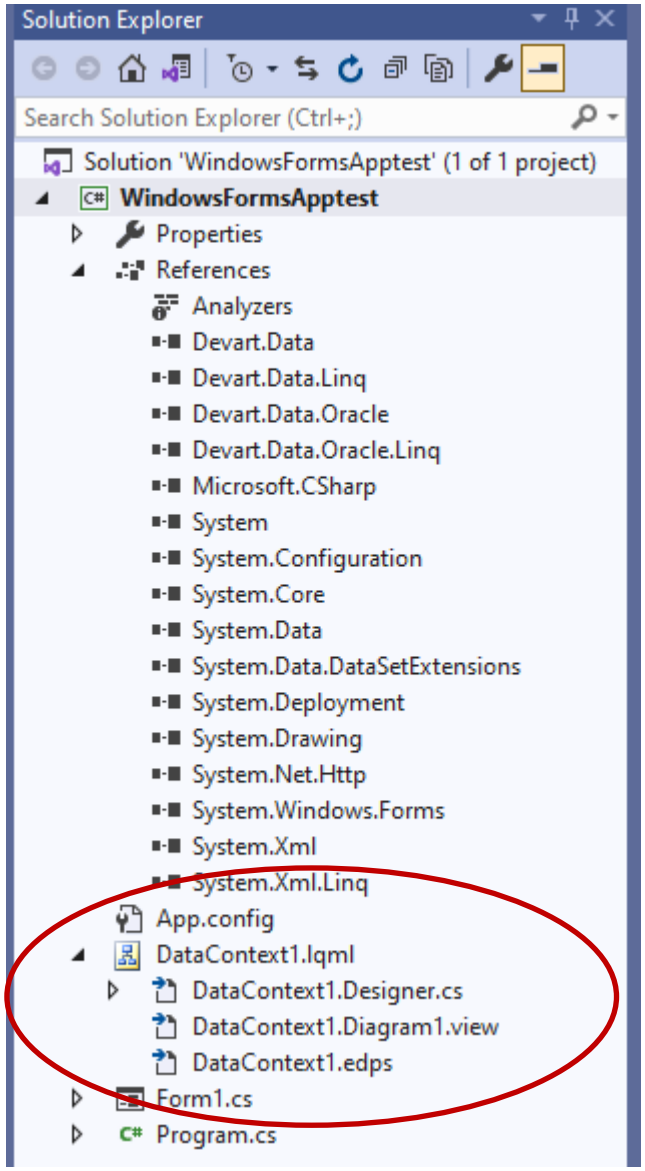

Entity Developer: Create Model Wizard
Model properties
On this page you can set the properties of the model.
Enter the name of the namespace that will contain your model:
IT123456Context
Enter name of DataContext:
IT123456DataContext
Preserve schema name in storage
Use database comments
Read-only computed columns
Detect Many-to-Many associations
Detect Table Per Type inheritances
Save connection settings in App.Config as:
IT123456DataContextConnectionString
< Back
Next >
Cancel

On finishing the wizard….



… the Model is added





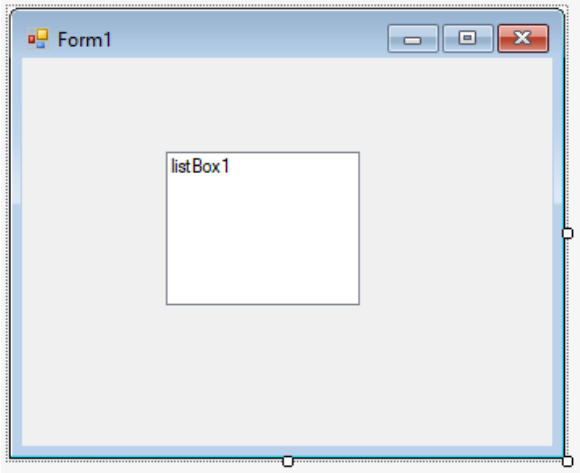
You will get the name of the namespace for accessing this model

```
using System;
using Devart.Data.Linq;
using Devart.Data.Linq.Mapping;
using System.Data;
using System.ComponentModel;
using System.Collections.Generic;
using System.Reflection;
using System.Linq;
using System.Linq.Expressions;

namespace IT123456Context
{

    [DatabaseAttribute(Name = "IT123456")]
    [ProviderAttribute(typeof(Devart.Data.Oracle.Linq.Provider.OracleDataProvi
    7 references
    public partial class IT123456DataContext : Devart.Data.Linq.DataContext
    {
        public static CompiledQueryCache compiledQueryCache = CompiledQueryCac
        private static MappingSource mappingSource = new Devart.Data.Linq.Mapp
```

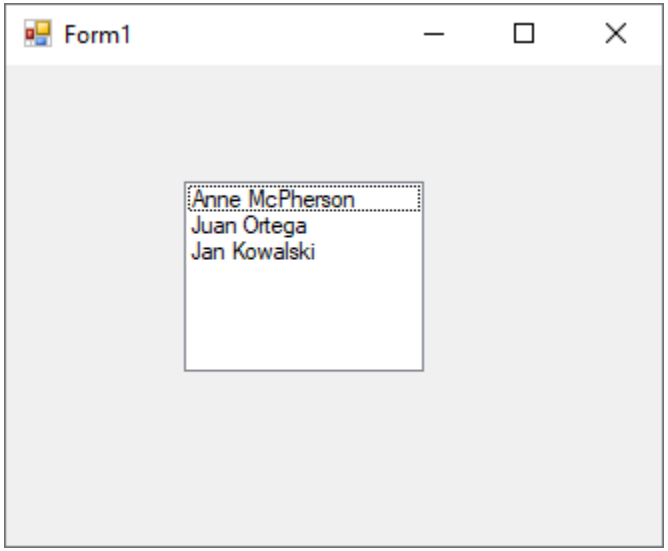
In the form, add a listbox -



Code

Run time

```
4   using System.Data;
5   using System.Drawing;
6   using System.Linq;
7   using System.Text;
8   using System.Threading.Tasks;
9   using System.Windows.Forms;
10  using IT123456Context;
11
12  namespace WindowsFormsAppwithLINQ
13  {
        3 references
14      public partial class Form1 : Form
15      {
            1 reference
16          public Form1()...
20
21
22          IT123456DataContext context;
            1 reference
23          private void Form1_Load(object sender, EventArgs e)
24          {
25              context = new IT123456DataContext();
26              var query = from c in context.CUSTOMER2S
27                          orderby c.CUSTNUM
28                          select c;
29              foreach (CUSTOMER2 cus in query)
30              {
31                  listBox1.Items.Add(cus.CUSTFNAME + " " +cus.CUSTLNAME);
32              }
33
34          }
35      }
36  }
```



Comments

Please note:

1. Slides 50 to 63 are just for demonstrating some examples in the Visual Studio environment (using both SQL and LINQ).
2. You will not be asked to use Visual Studio in any assessment (i.e., assignment or test).
3. These examples just demonstrate how the two queries can be used.